Relational Separation Logic for Compiler Verification

Gabriel Desfrene
Supervised by Xavier Leroy and François Pottier

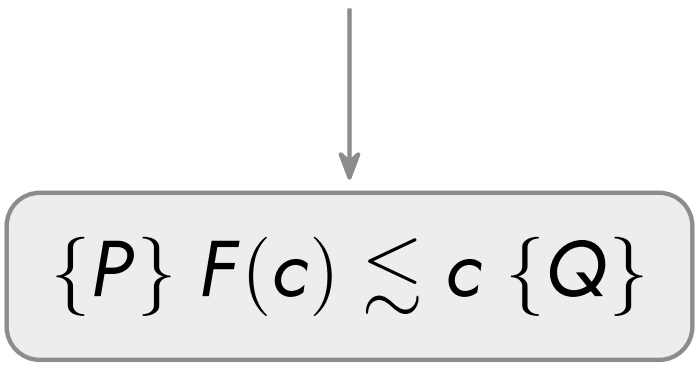
September 2026

Research Goals

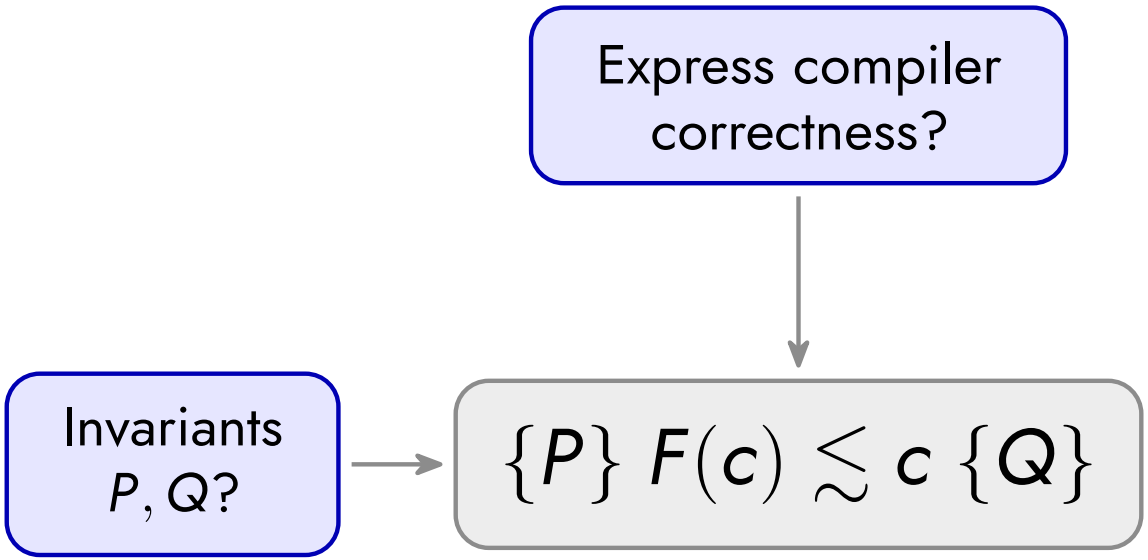
$$\{P\}\ F(c) \lesssim c\ \{Q\}$$

Research Goals

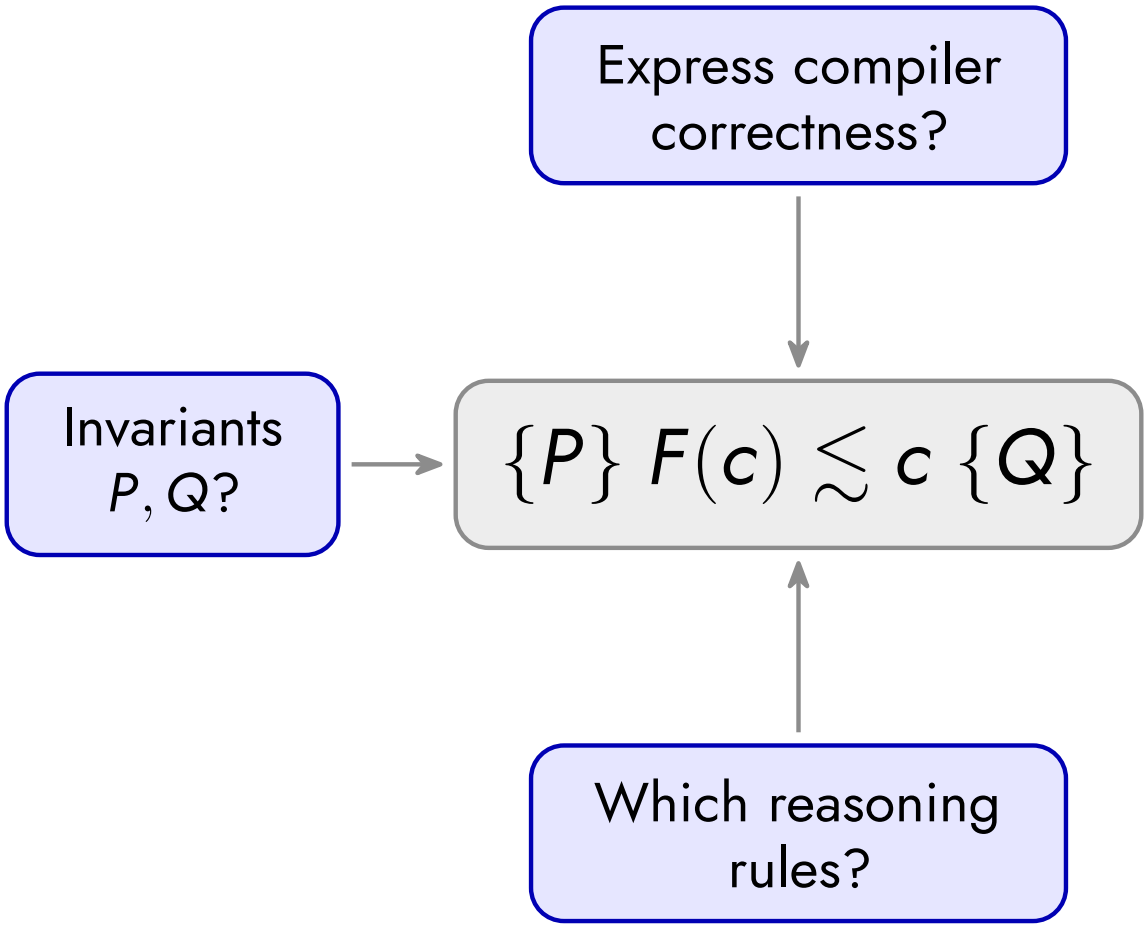
Express compiler correctness?



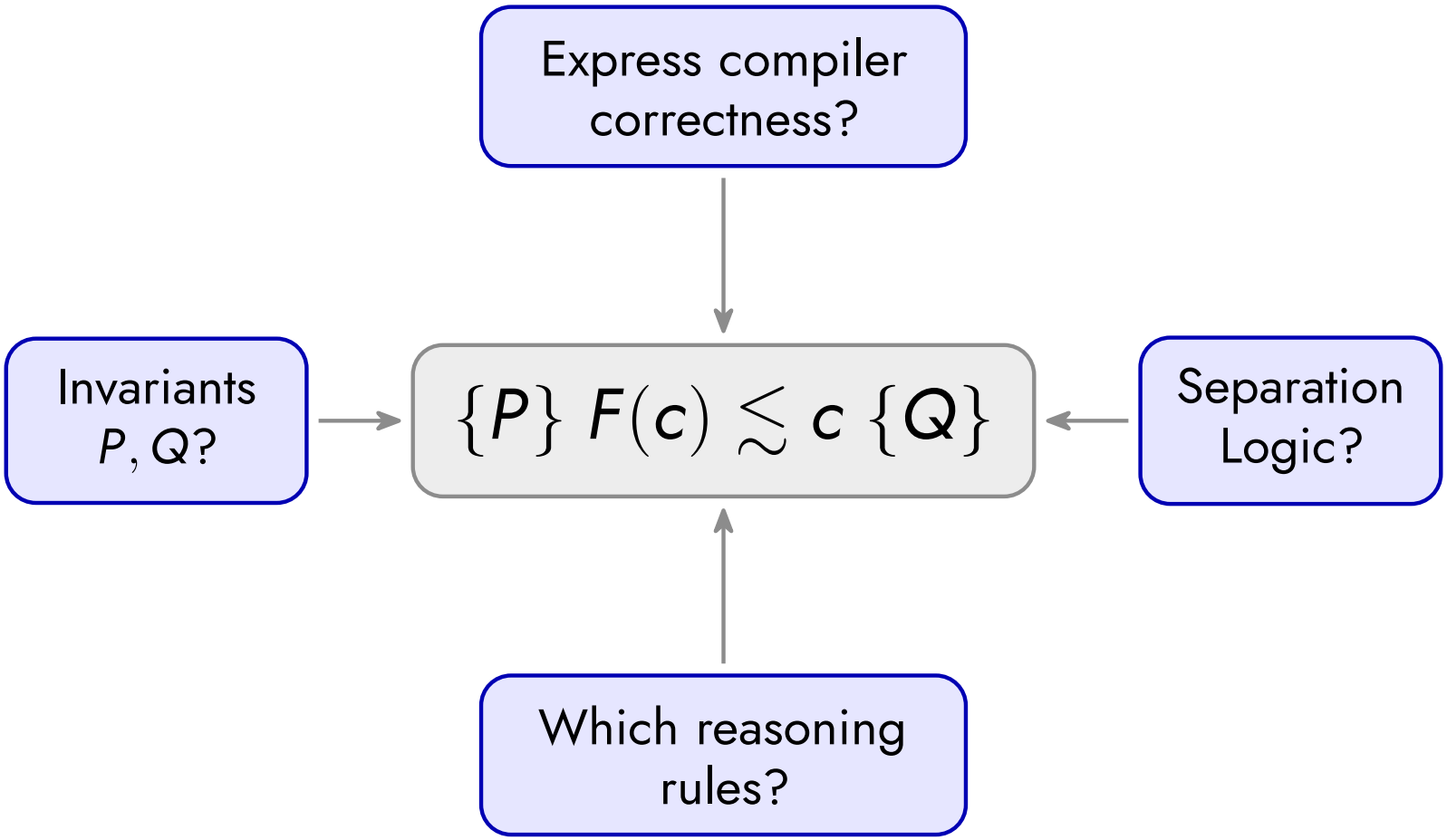
Research Goals



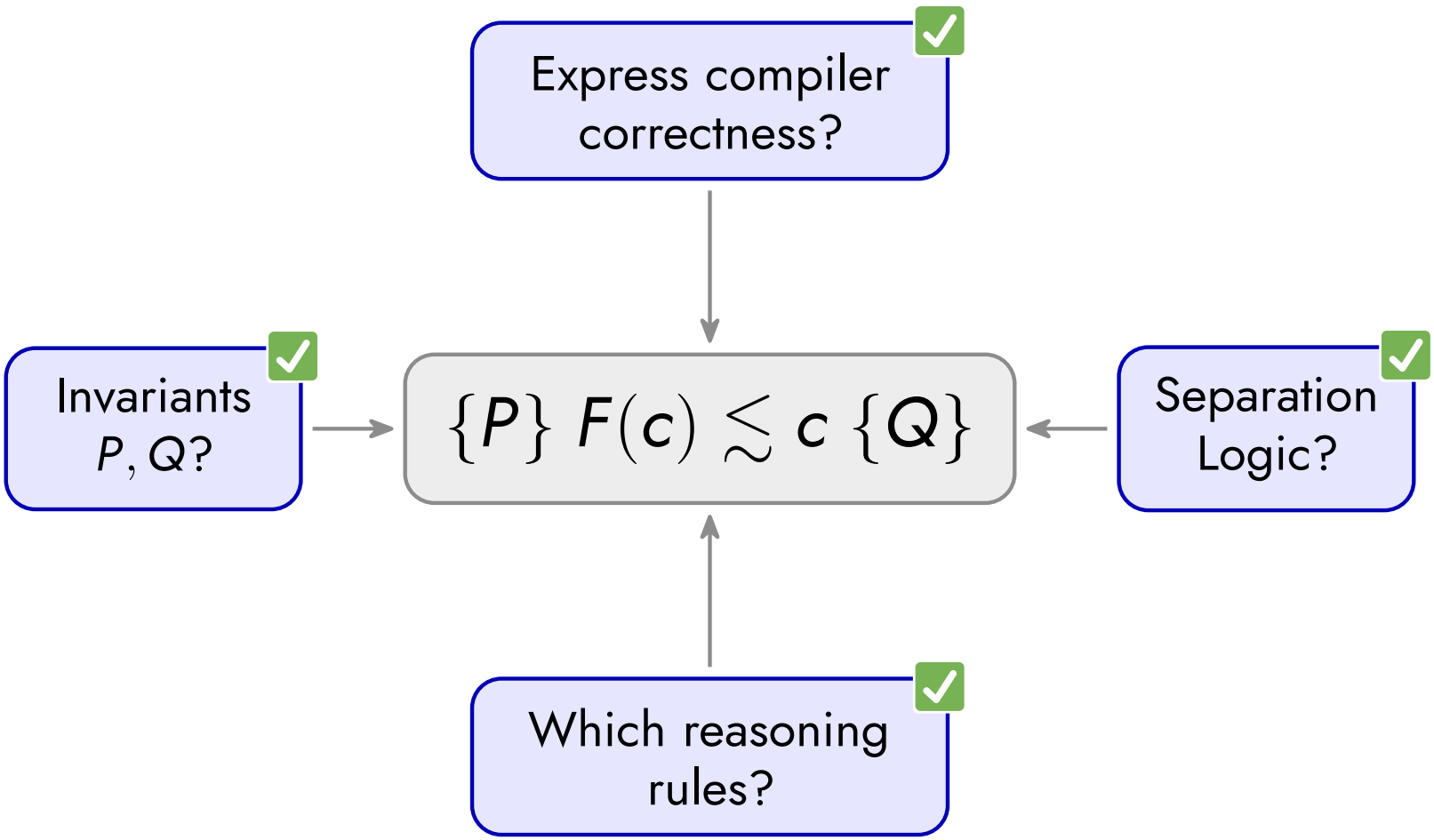
Research Goals



Research Goals



Research Goals



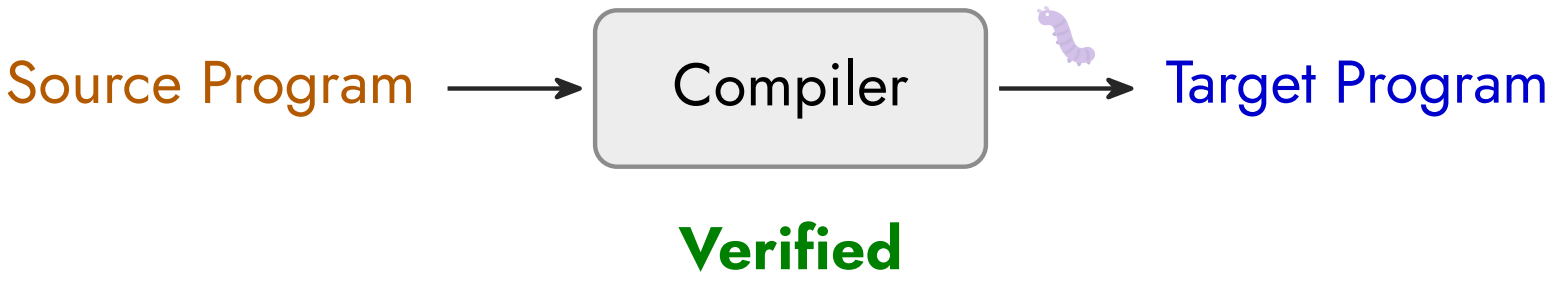
Context

Compilers Can Introduce Bugs

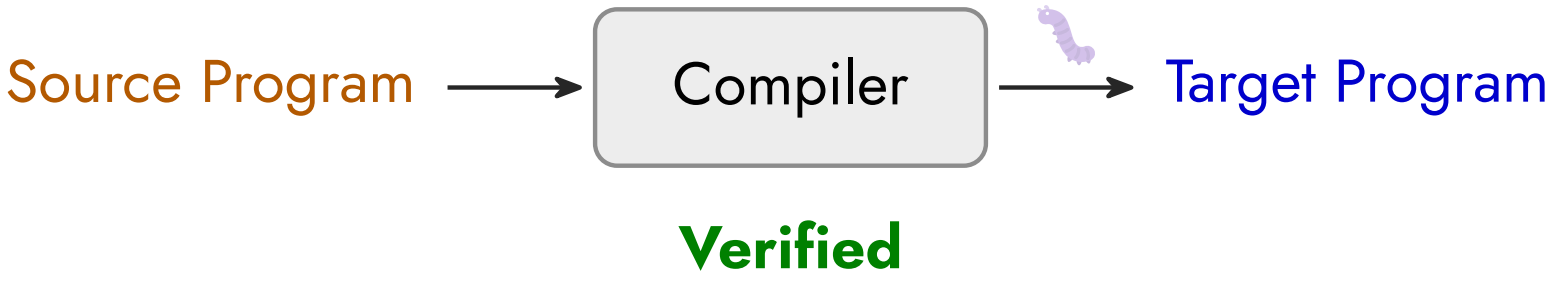
Compilers Can Introduce Bugs

Compilers Can Introduce Bugs

Compilers Can Introduce Bugs



Compilers Can Introduce Bugs



CakeML
ML → machine

CompCert
C → assembly

Jasmin
crypto → machine

Behavioral Refinement

Behavioral Refinement

$$b \in \{\texttt{Term}(v, m) \mid \texttt{Div} \mid \lightning\}$$

Behavioral Refinement

$$b \in \{\mathtt{Term}(v, m) \mid \mathtt{Div} \mid \lightning\}$$

Behavioral Refinement

$$b \in \{\mathtt{Term}(v, m) \mid \mathtt{Div} \mid \lightning\}$$

$b \in T \iff b \in S$

$$\mathtt{f}\big((\mathtt{print}(\texttt{"A"}); 1), (\mathtt{print}(\texttt{"B"}); 2)\big)$$
$$\downarrow$$
$$\mathtt{print}(\texttt{"A"}); \mathtt{print}(\texttt{"B"}); \mathtt{f}(1, 2)$$

Behavioral Refinement

$$b \in \{\mathtt{Term}(v, m) \mid \mathtt{Div} \mid \lightning\}$$

$b \in T \iff b \in S$

$\mathtt{f}((\mathtt{print}(\mathtt{"A"}); 1), (\mathtt{print}(\mathtt{"B"}); 2))$
$\downarrow$
$\mathtt{print}(\mathtt{"A"}); \mathtt{print}(\mathtt{"B"}); \mathtt{f}(1, 2)$

$b \in T \implies b \in S$

Behavioral Refinement

$$b \in \{\mathtt{Term}(v, m) \mid \mathtt{Div} \mid \lightning\}$$

$b \in T \iff b \in S$	$\mathtt{f}((\mathtt{print}(\texttt{"A"}); 1), (\mathtt{print}(\texttt{"B"}); 2)) \downarrow \mathtt{print}(\texttt{"A"}); \mathtt{print}(\texttt{"B"}); \mathtt{f}(1, 2)$
$b \in T \implies b \in S$	$x := y/y \quad \to \quad x := 1$

Behavioral Refinement

$$b \in \{\mathtt{Term}(v, m) \mid \mathtt{Div} \mid \lightning\}$$

$b \in T \iff b \in S$	$\mathtt{f}((\mathtt{print}(\mathtt{"A"}); 1), (\mathtt{print}(\mathtt{"B"}); 2)) \downarrow \mathtt{print}(\mathtt{"A"}); \mathtt{print}(\mathtt{"B"}); \mathtt{f}(1, 2)$
$b \in T \implies b \in S$	$x := y/y \quad \to \quad x := 1$

$T \preceq S \triangleq$

Behavioral Refinement

$$b \in \{\mathtt{Term}(v, m) \mid \mathtt{Div} \mid \lightning\}$$

$b \in T \iff b \in S$	$\mathtt{f}((\mathtt{print}(\texttt{"A"}); 1), (\mathtt{print}(\texttt{"B"}); 2)) \downarrow \mathtt{print}(\texttt{"A"}); \mathtt{print}(\texttt{"B"}); \mathtt{f}(1, 2)$
$b \in T \implies b \in S$	$x := y/y \quad \to \quad x := 1$

$T \preceq S \triangleq$

$\lightning \in S$

or

$b \in T \implies b \in S$

Simulation Relation

$$t_0 \to t_1 \to \cdots \to \texttt{Term}(v, m)$$

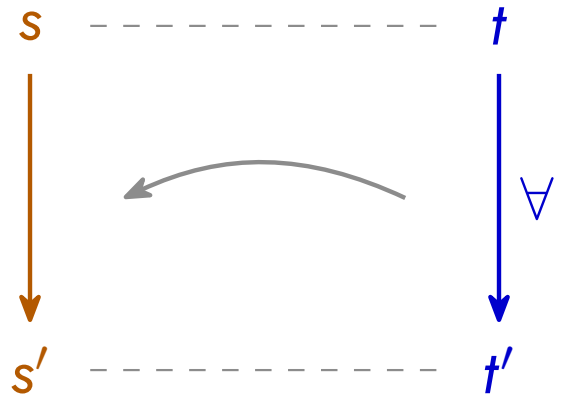
$$\Downarrow$$

$$s_0 \to s_1 \to \cdots \to \texttt{Term}(v, m)$$

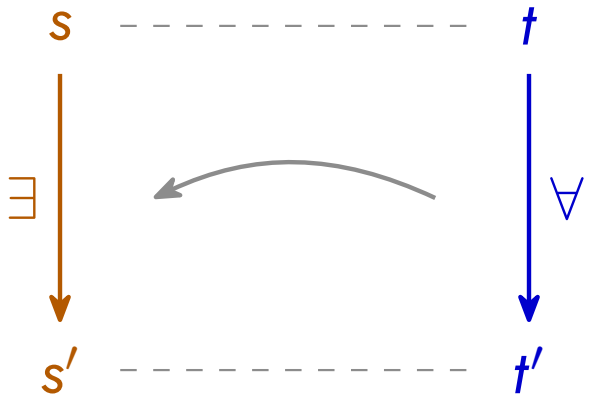
Simulation Relation

Simulation Relation

Simulation Relation

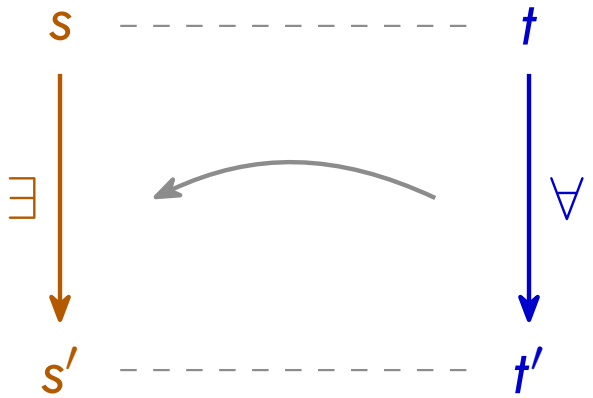


Simulation Relation



Simulation Relation

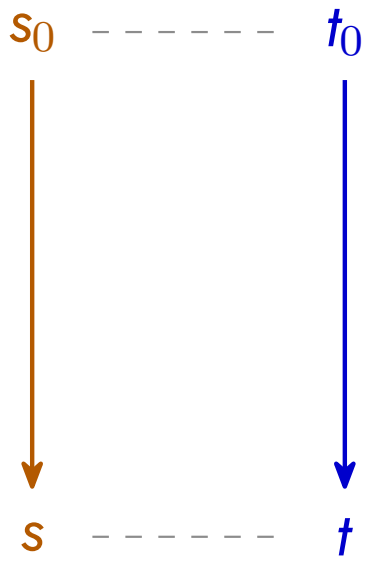
Backward Simulation



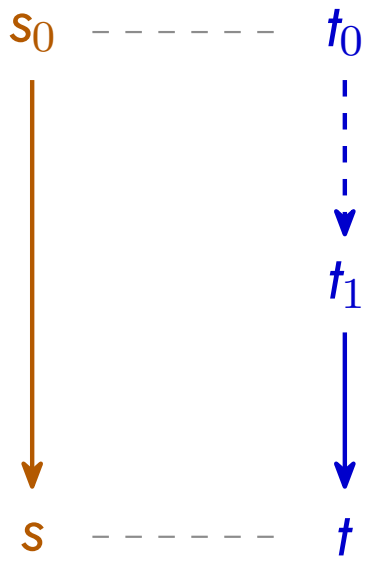
$$\Downarrow$$

$$T \preceq S$$

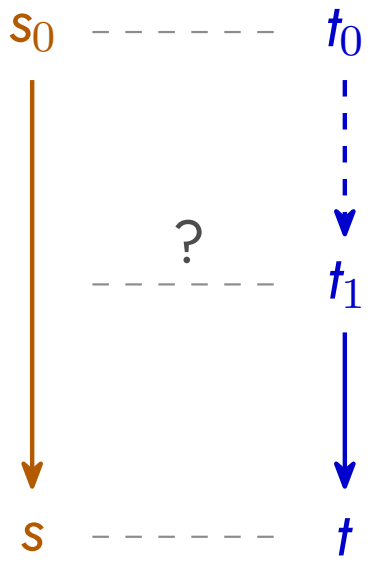
The Stuttering Problem



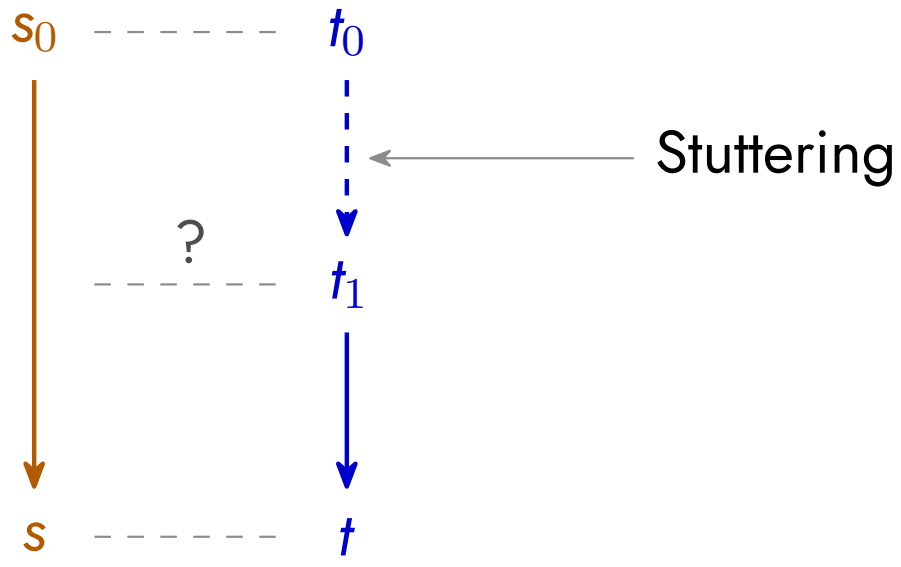
The Stuttering Problem



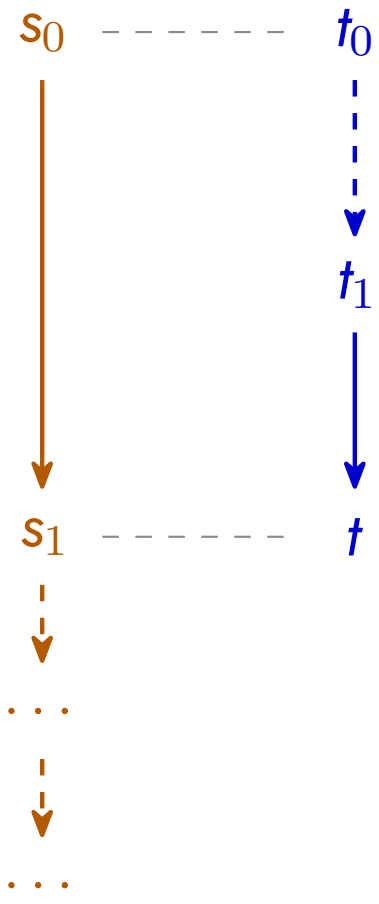
The Stuttering Problem



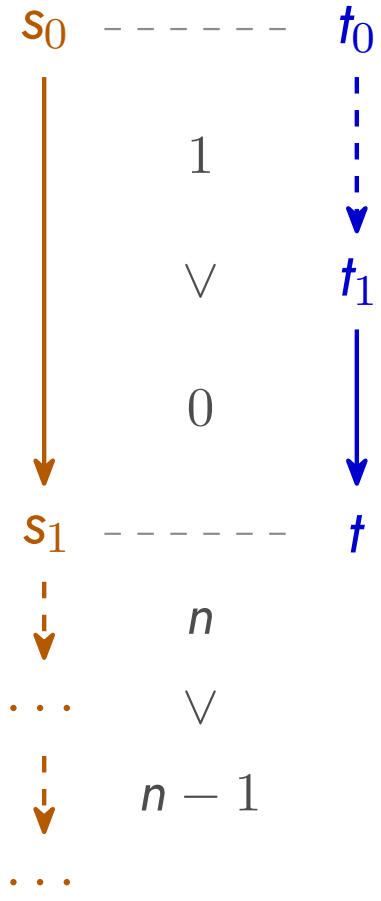
The Stuttering Problem



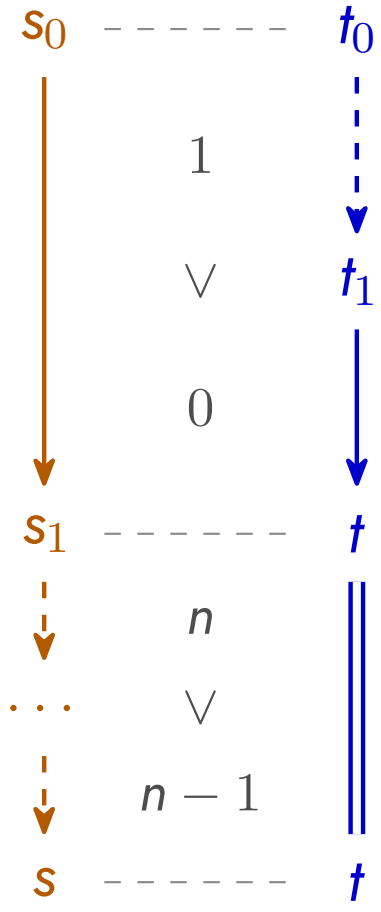
The Stuttering Problem



The Stuttering Problem



The Stuttering Problem



Limitations of Existing Approaches

Limitations of Existing Approaches

	Non-Determinism	

Limitations of Existing Approaches

	Non-Determinism	
CakeML	Not Supported	

Limitations of Existing Approaches

	Non-Determinism	
CakeML	Not Supported	
CompCert	External Only	

Limitations of Existing Approaches

	Non-Determinism	
CakeML	Not Supported	
	Backward Simulation	
CompCert	External Only	

Limitations of Existing Approaches

	Non-Determinism	Memory Reasoning
CakeML	Not Supported	
	Backward Simulation	
CompCert	External Only	

Limitations of Existing Approaches

	Non-Determinism	Memory Reasoning
CakeML	Not Supported	Global Invariants
CompCert	External Only	Global Invariants

Backward Simulation

Limitations of Existing Approaches

	Non-Determinism	Memory Reasoning
CakeML	Not Supported	Global Invariants
	Backward Simulation	**Separation Logic**
CompCert	External Only	Global Invariants

Contributions

Contributions

Backward Simulation

inspired by "Stuttering For Free"

Contributions

Relational Separation Logic

inspired by "Simuliris"

Backward Simulation

inspired by "Stuttering For Free"

Contributions

Language Dependent

Language Independent

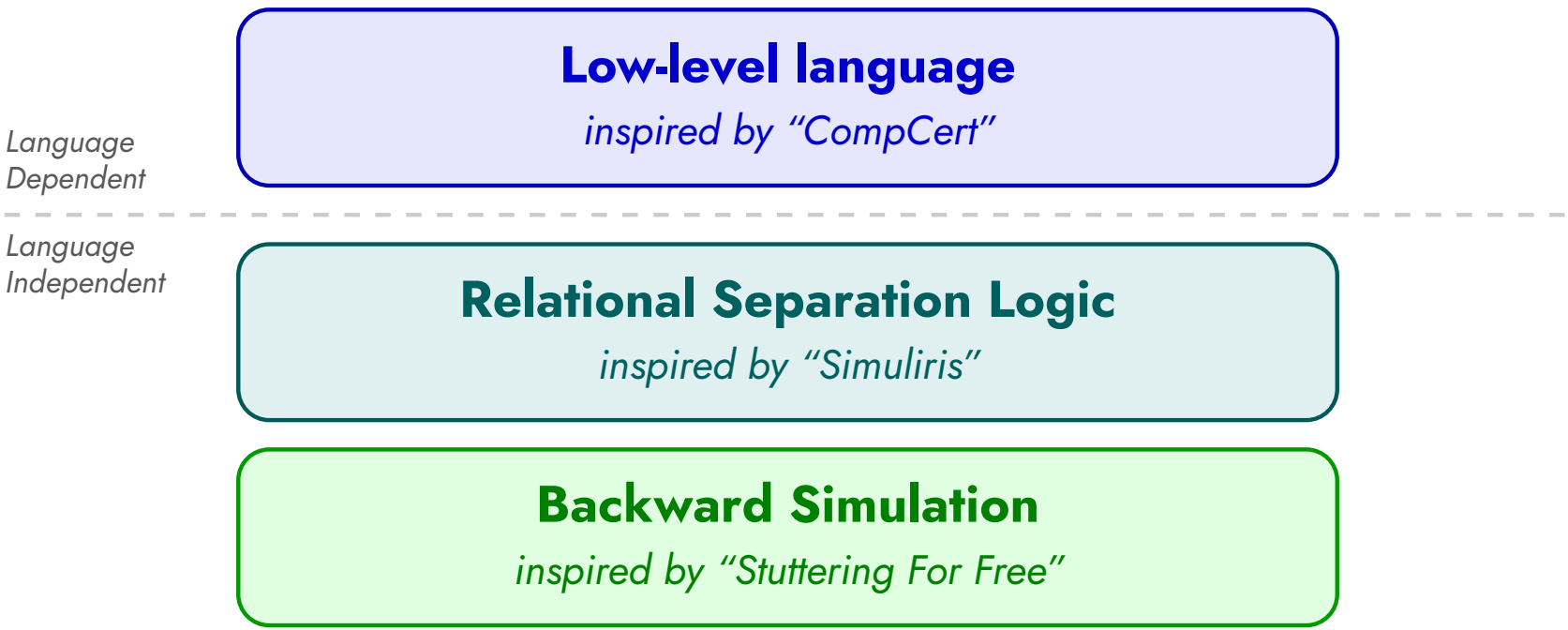
Relational Separation Logic

inspired by "Simuliris"

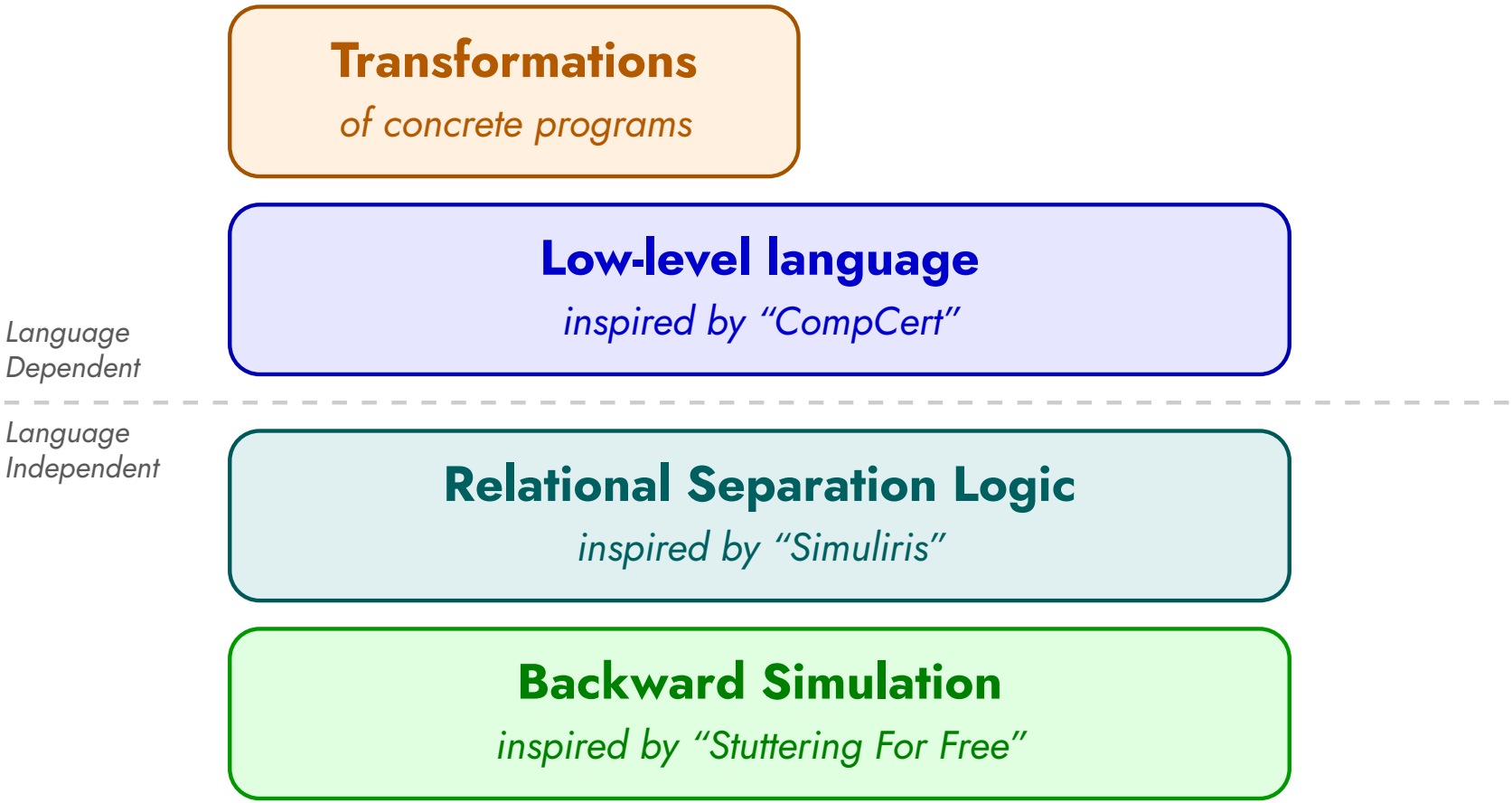
Backward Simulation

inspired by "Stuttering For Free"

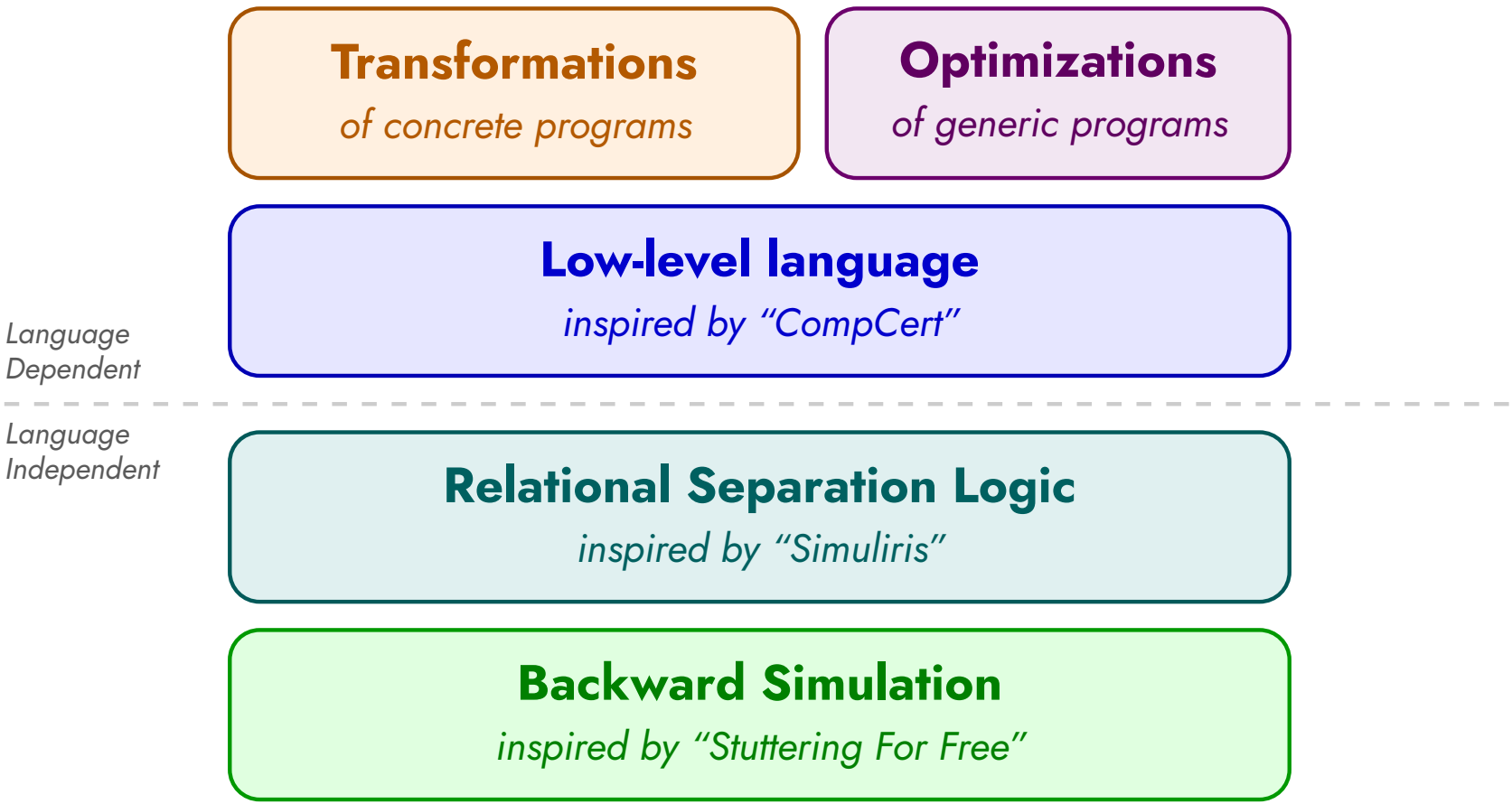
Contributions

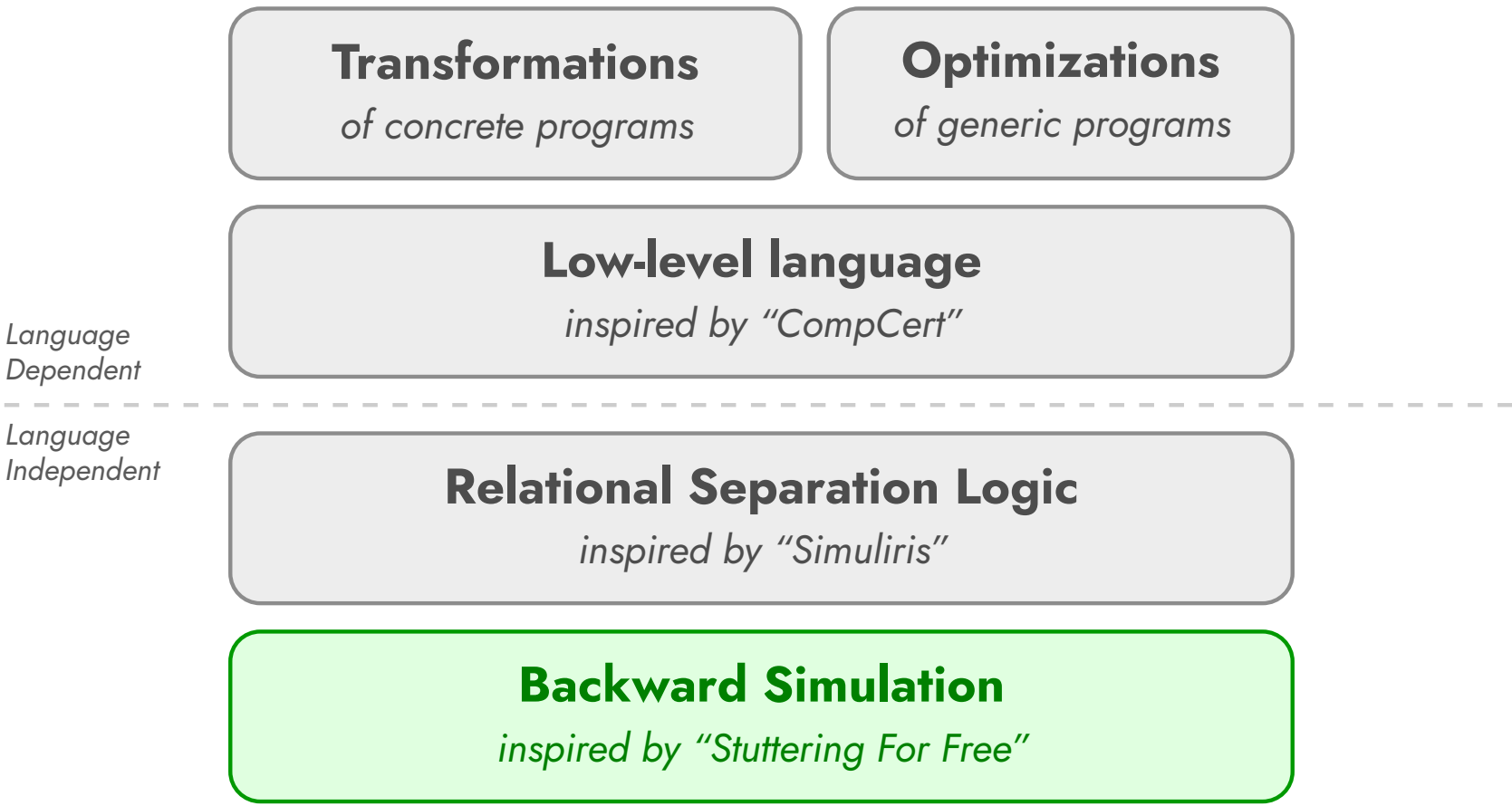


Contributions



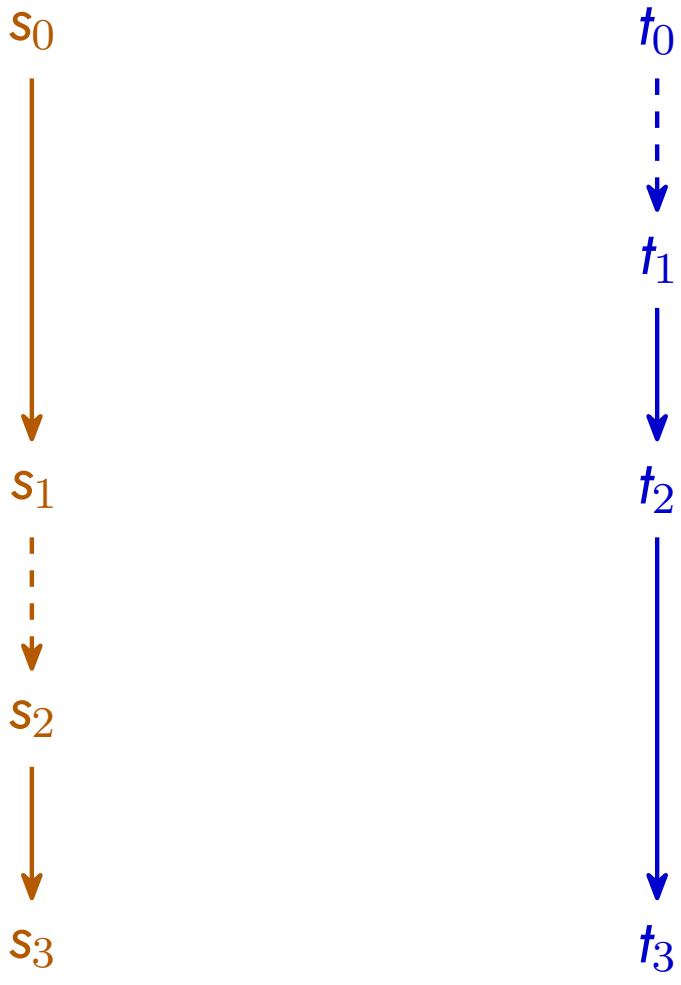
Contributions



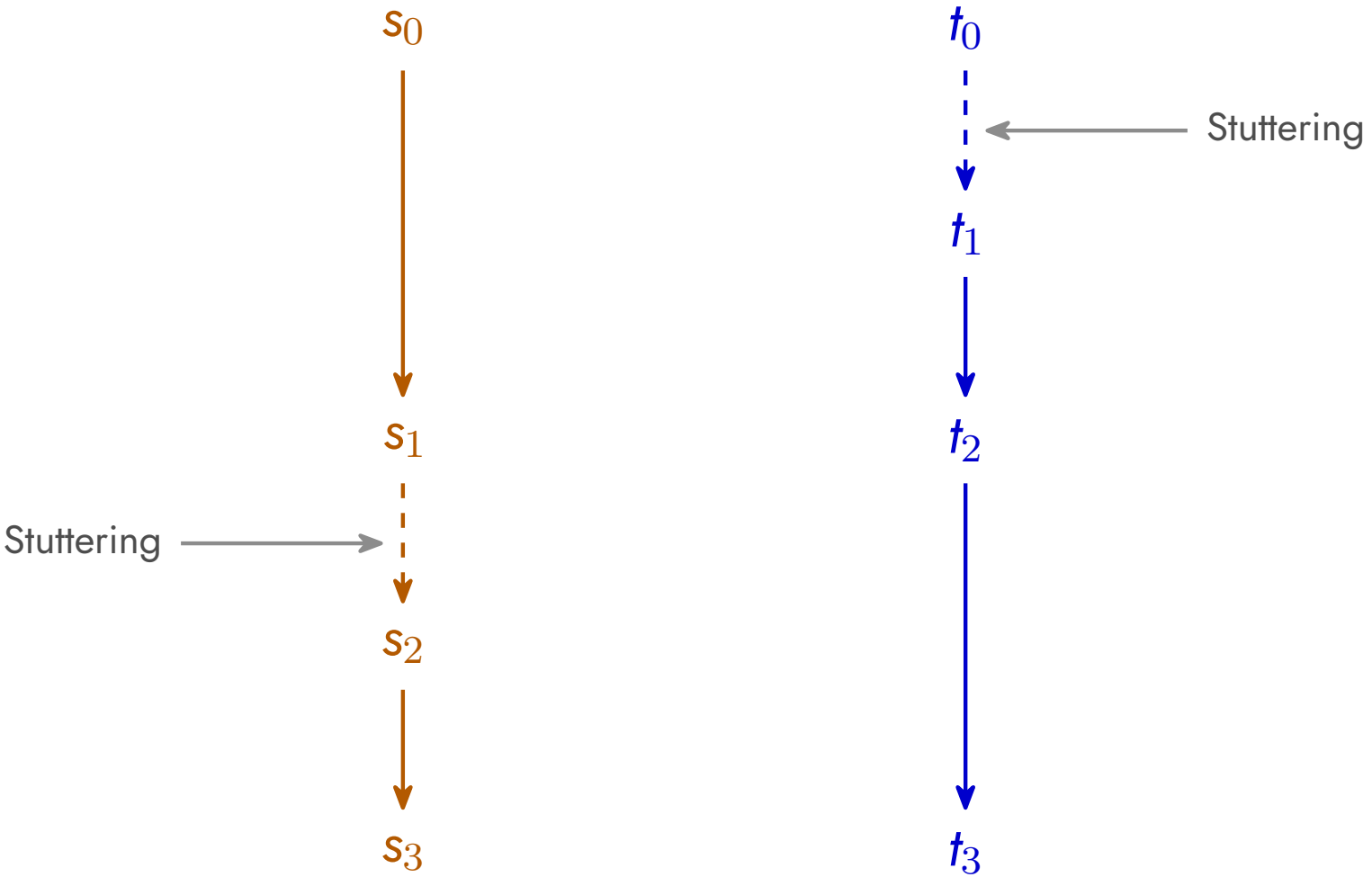

Transformations
of concrete programs
Optimizations
of generic programs
Low-level language
inspired by "CompCert"
Language Dependent
Language Independent
Relational Separation Logic
inspired by "Simuliris"
Backward Simulation
inspired by "Stuttering For Free"

The Free Simulation

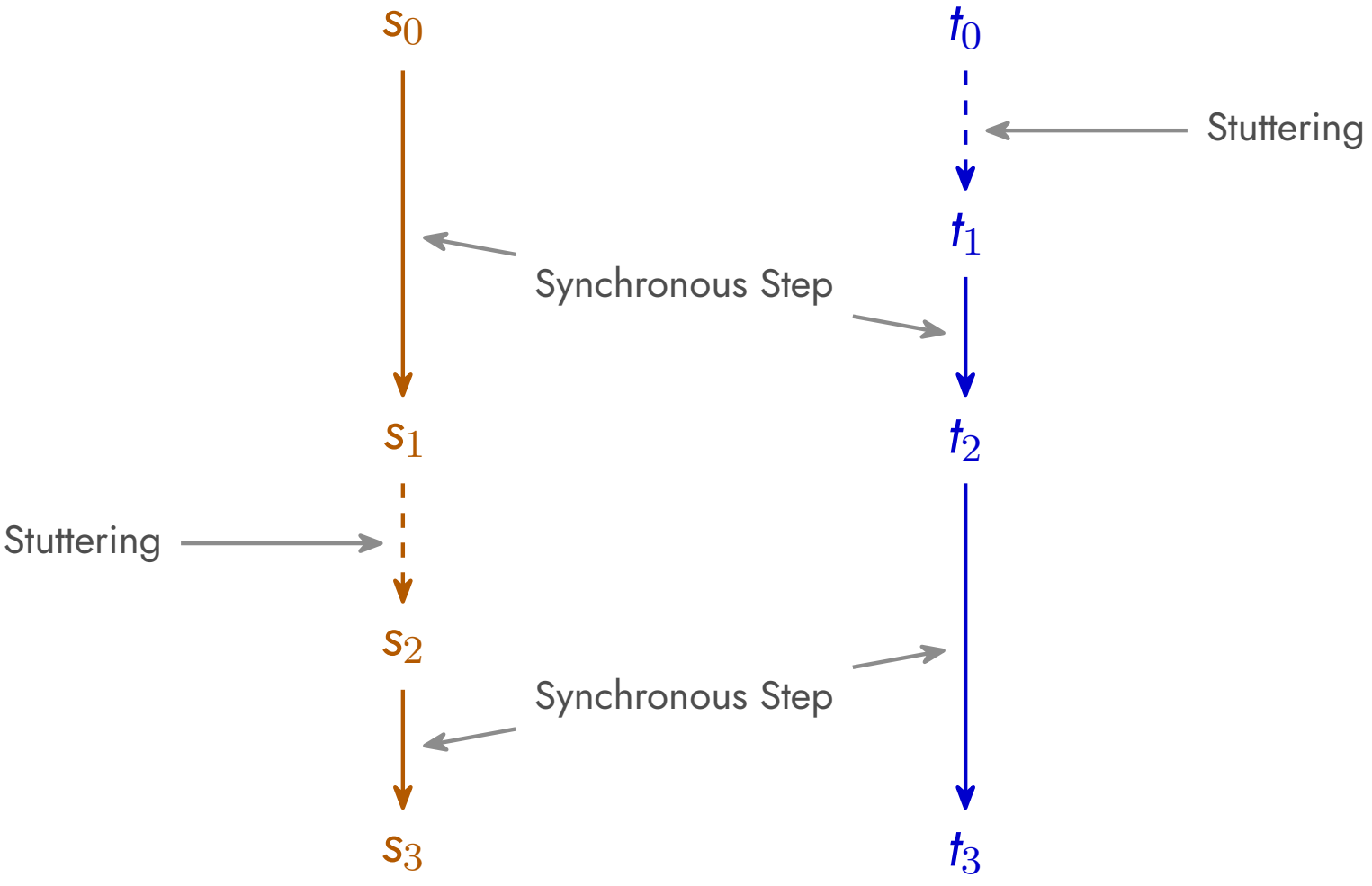
The Free Simulation



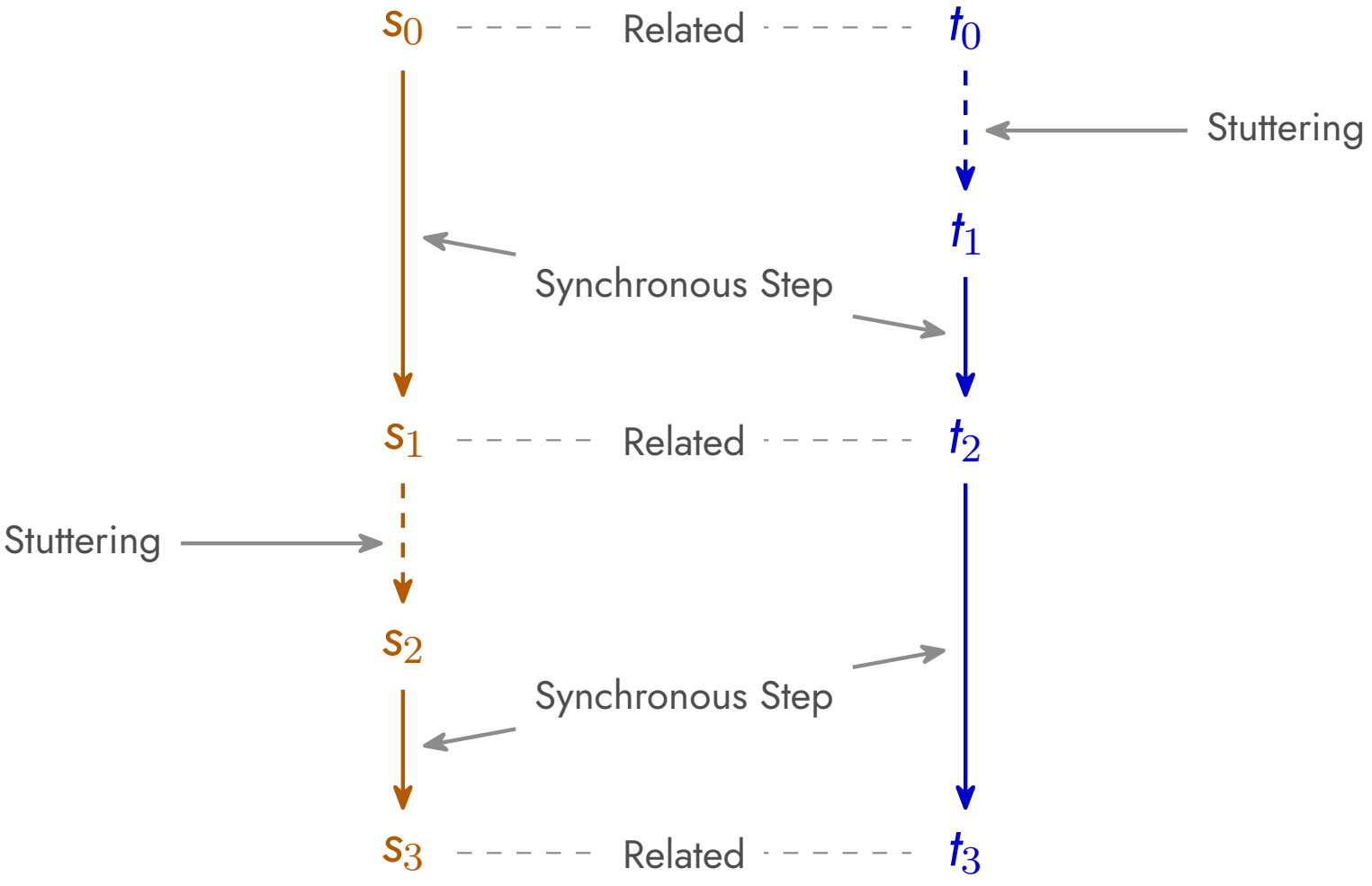
The Free Simulation



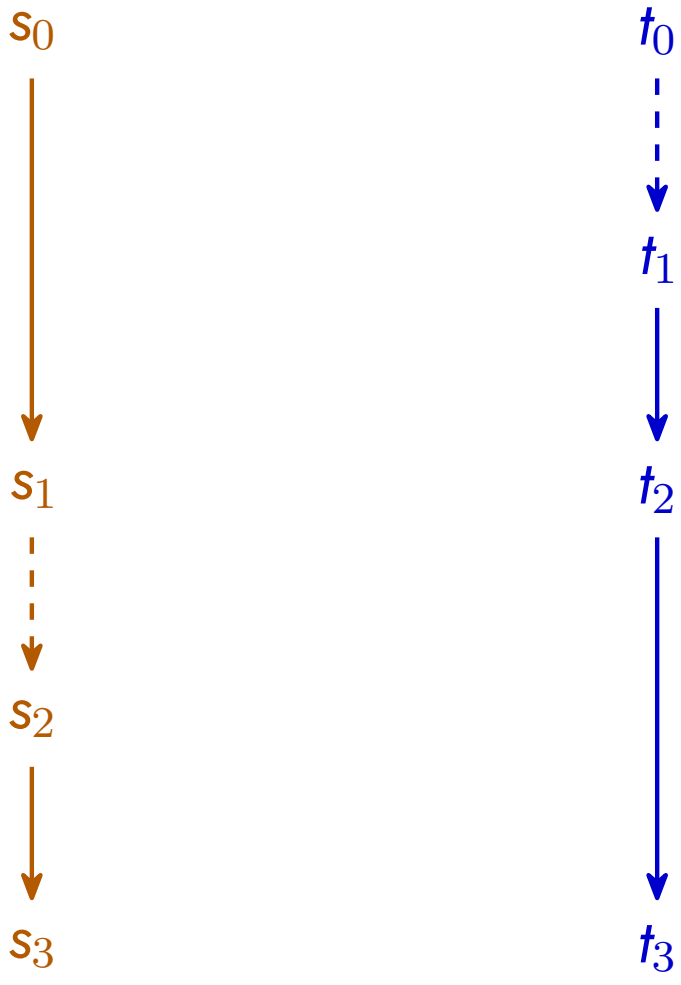
The Free Simulation



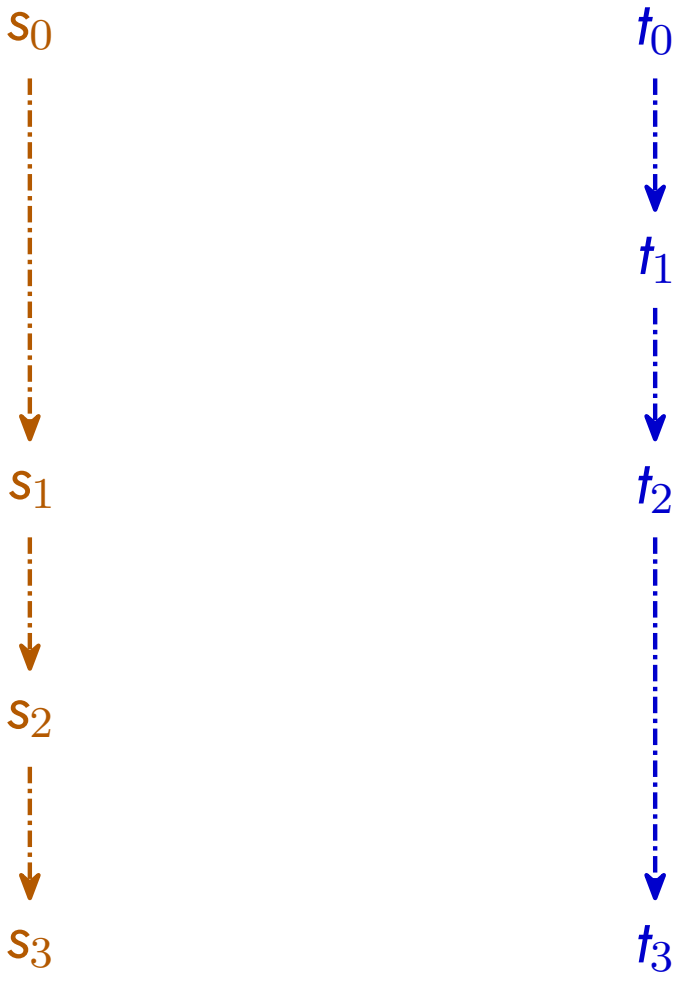
The Free Simulation



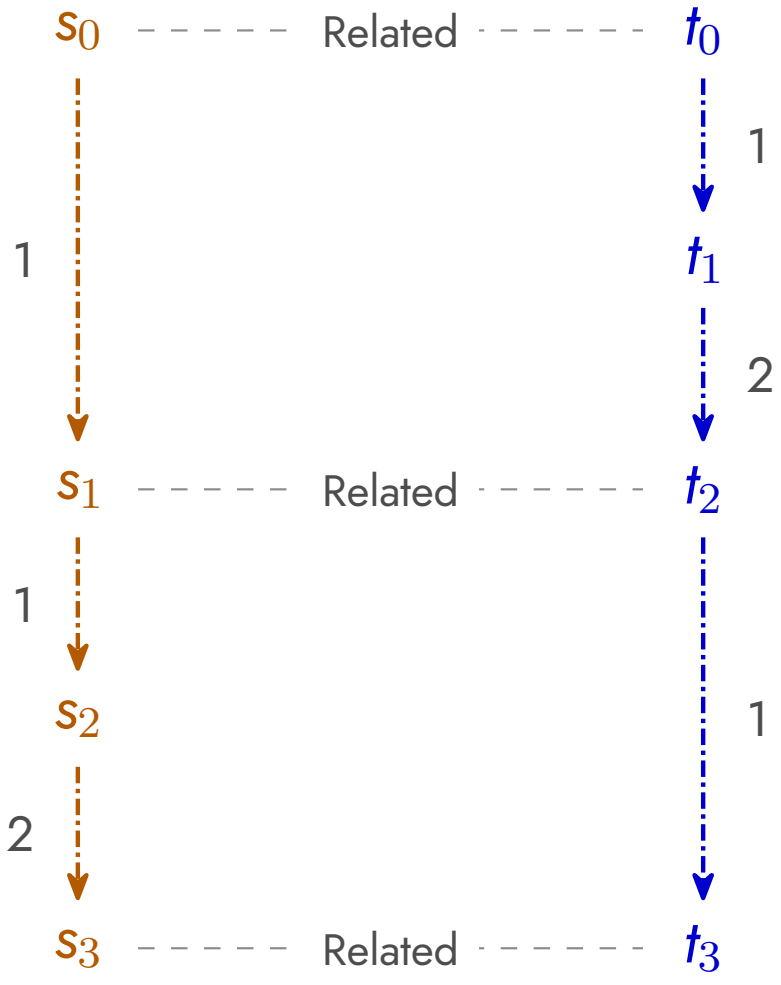
The Free Simulation



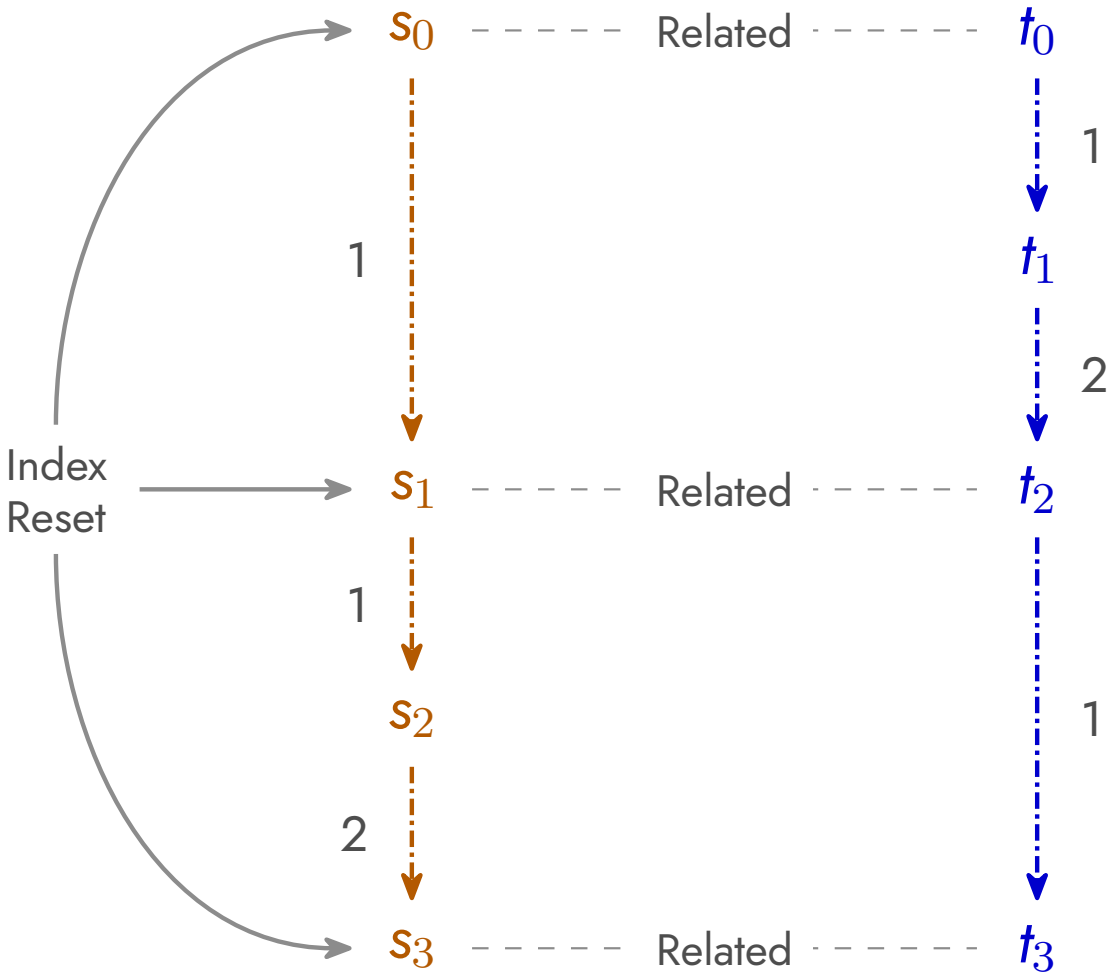
The Free Simulation



The Free Simulation



The Free Simulation



Theoretical Guarantees

Theoretical Guarantees

Soundness:

$$\forall i\, j, \qquad \langle T, i\rangle \sim_{\mathsf{free}} \langle S, j\rangle \implies T \preceq S$$

Theoretical Guarantees

Soundness:

$$\forall i\, j, \qquad \langle T, i\rangle \sim_{\text{free}} \langle S, j\rangle \implies T \preceq S$$

Equivalence:

$$(\exists i\, j,\ \langle T, i\rangle \sim_{\text{free}} \langle S, j\rangle) \iff T \sim_{\text{explicit}} S$$

Theoretical Guarantees

Soundness:

$$\forall i\, j, \qquad \langle T, i\rangle \sim_{\text{free}} \langle S, j\rangle \implies T \preceq S$$

Equivalence:

$$(\exists i\, j,\ \langle T, i\rangle \sim_{\text{free}} \langle S, j\rangle) \iff T \sim_{\text{explicit}} S$$

Index Irrelevance:

$$(\exists i\, j,\ \langle T, i\rangle \sim_{\text{free}} \langle S, j\rangle) \implies \forall i\, j, \quad \langle T, i\rangle \sim_{\text{free}} \langle S, j\rangle$$

Asymmetric Reasoning

Asymmetric Reasoning

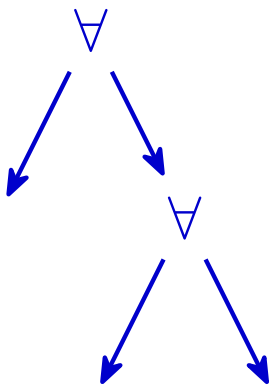
Target	Source

Asymmetric Reasoning

Target	Source
😈 nondeterminism	

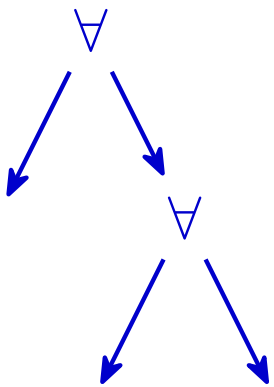
Asymmetric Reasoning

Target	Source
😈 nondeterminism	



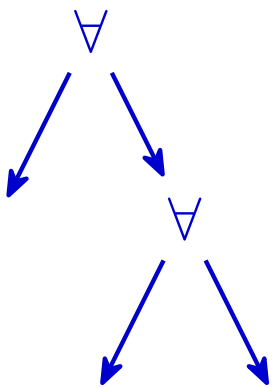
Asymmetric Reasoning

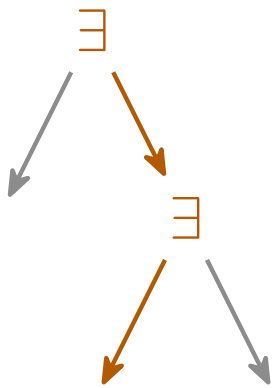
Target	Source
😈 nondeterminism	😇 nondeterminism



Asymmetric Reasoning

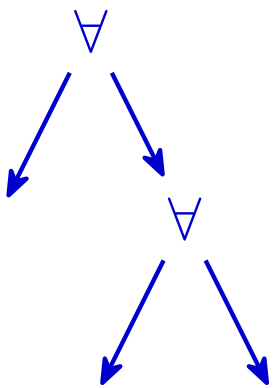
Target	Source
😈 nondeterminism	😇 nondeterminism

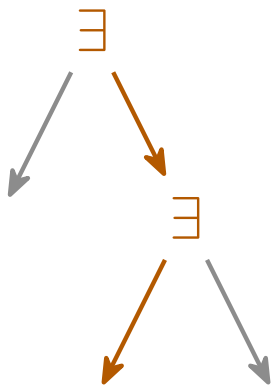




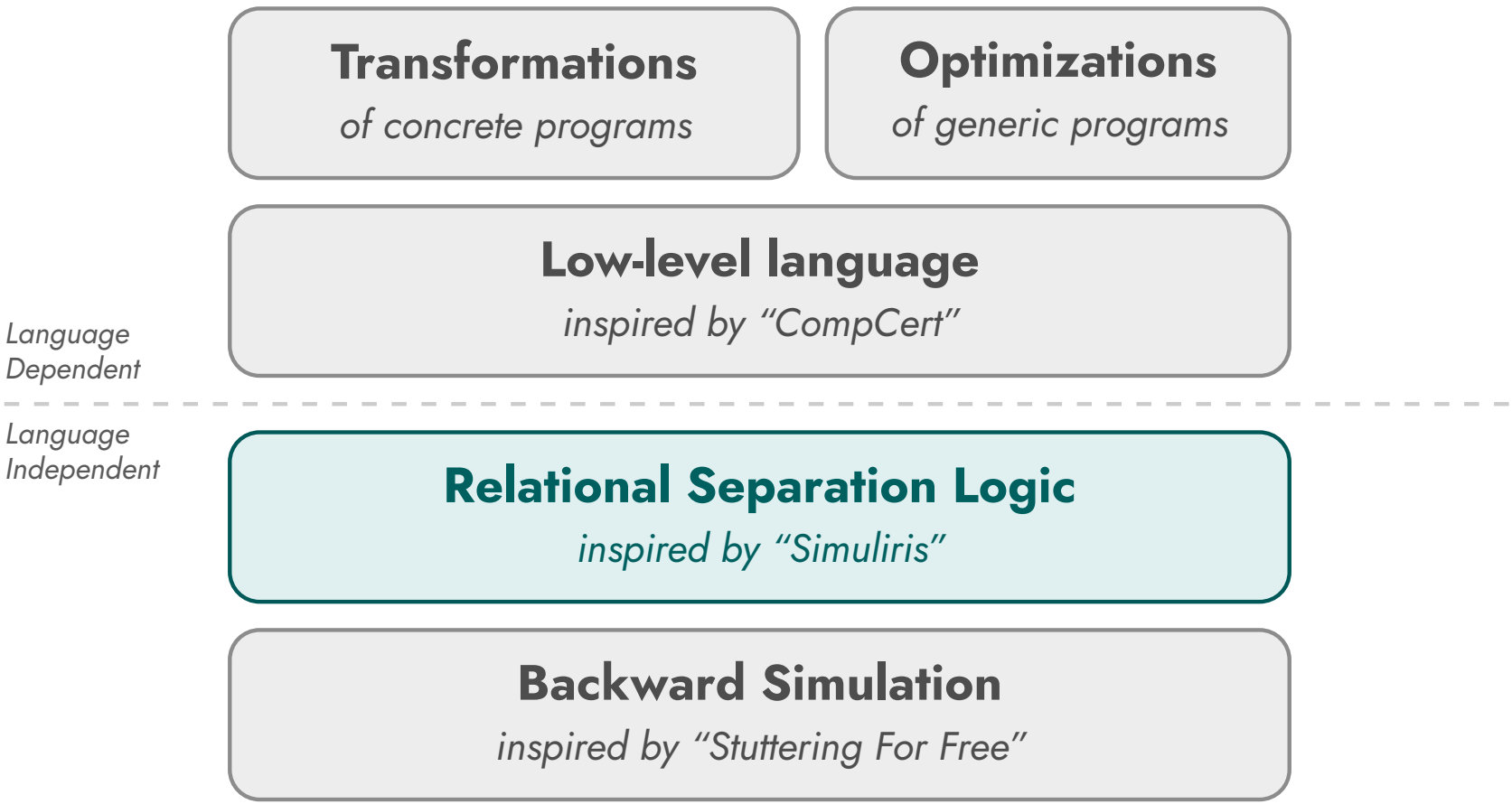
Asymmetric Reasoning

Target	Source
😈 nondeterminism	😇 nondeterminism





$$\lightning \in S \implies T \sim_{\text{free}} S$$


Transformations
of concrete programs
Optimizations
of generic programs
Low-level language
inspired by "CompCert"
Language Dependent
Language Independent
Relational Separation Logic
inspired by "Simuliris"
Backward Simulation
inspired by "Stuttering For Free"

Relational Hoare Quadruples

Relational Hoare Quadruples

$$\{P\}\ f_t \lesssim f_s\ \{Q\}$$

Relational Hoare Quadruples

$$\{P\}\ f_t \lesssim f_s\ \{Q\}$$

$$\triangleq P \twoheadrightarrow \mathsf{FreeSim}\,(f_t, f_s, Q)$$

Relational Hoare Quadruples

$$\{P\}\ f_t \lesssim f_s\ \{Q\}$$

$$\triangleq P \twoheadrightarrow \mathsf{FreeSim}\,(f_t, f_s, Q)$$

If $P\,(\vec{v_t}, \vec{v_s})$:

Relational Hoare Quadruples

$$\{P\}\ f_t \lesssim f_s\ \{Q\}$$

$$\triangleq P \twoheadrightarrow \mathsf{FreeSim}\,(f_t, f_s, Q)$$

If $P\,(\vec{v_t}, \vec{v_s})$:

$$\lightning \in f_s$$

Relational Hoare Quadruples

$$\{P\}\ f_t \lesssim f_s\ \{Q\}$$

$$\triangleq P \twoheadrightarrow \mathsf{FreeSim}\,(f_t, f_s, Q)$$

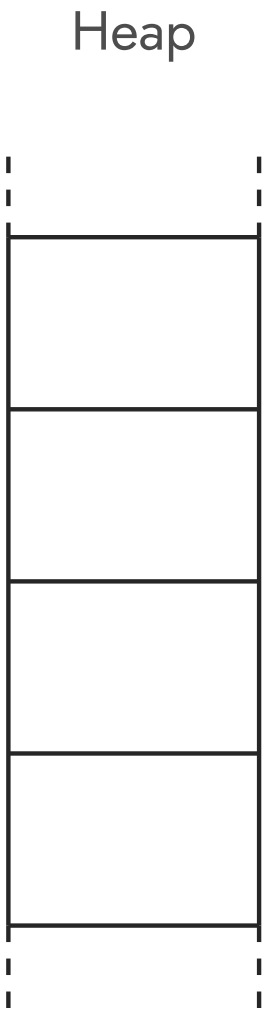
If $P\,(\vec{v_t}, \vec{v_s})$:

$$\lightning \in f_s$$

or

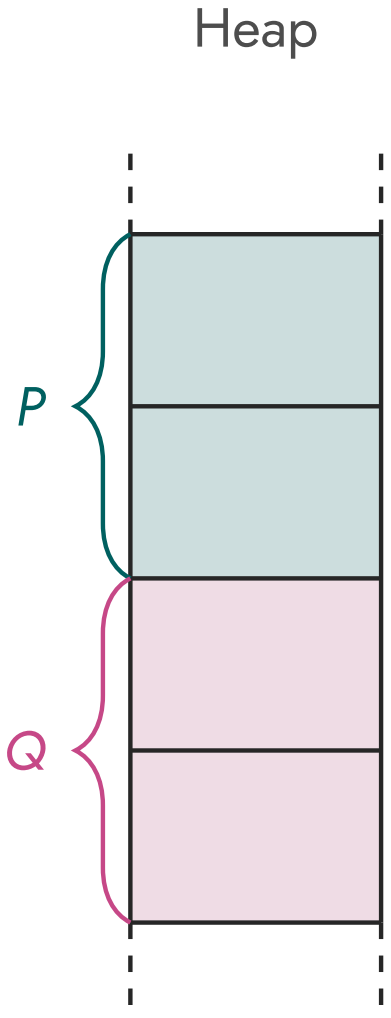
$$\begin{cases} \mathtt{Term}(v_t, m_t) \in f_t \implies \mathtt{Term}(v_s, m_s) \in f_s \text{ with } Q\,(v_t, v_s, m_t, m_s) \\ \mathtt{Div} \in f_t \implies \mathtt{Div} \in f_s \end{cases}$$

Relational Separation Logic



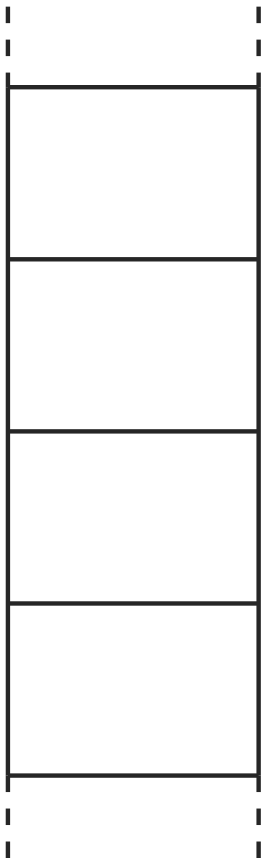
Relational Separation Logic

$P * Q \implies$

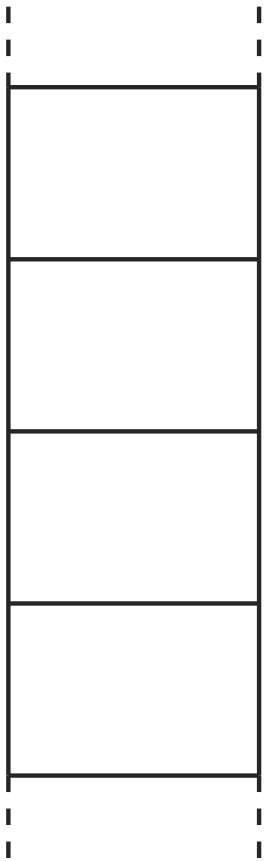


Relational Separation Logic

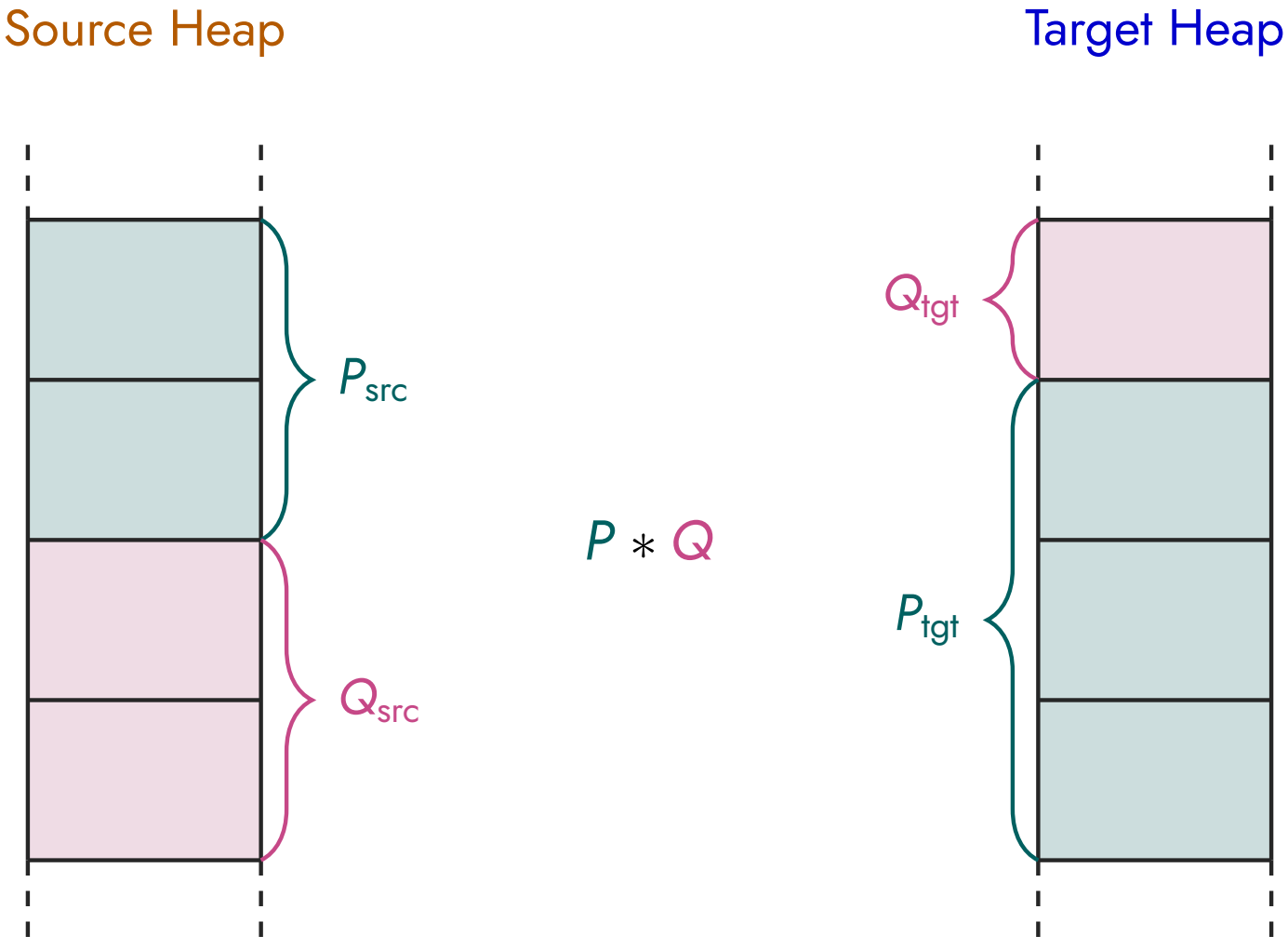
Source Heap



Target Heap



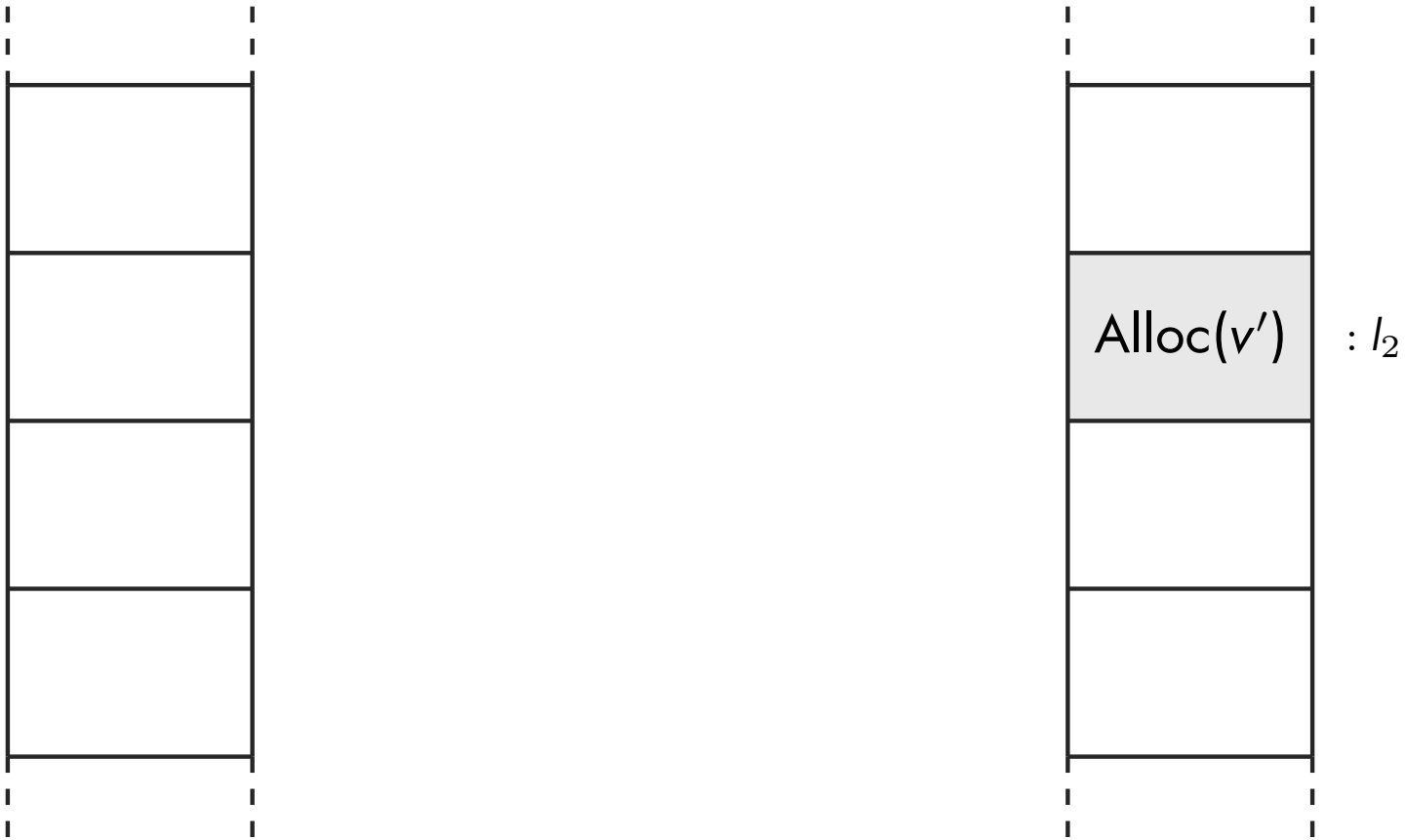
Relational Separation Logic



Relational Separation Logic

Source Heap

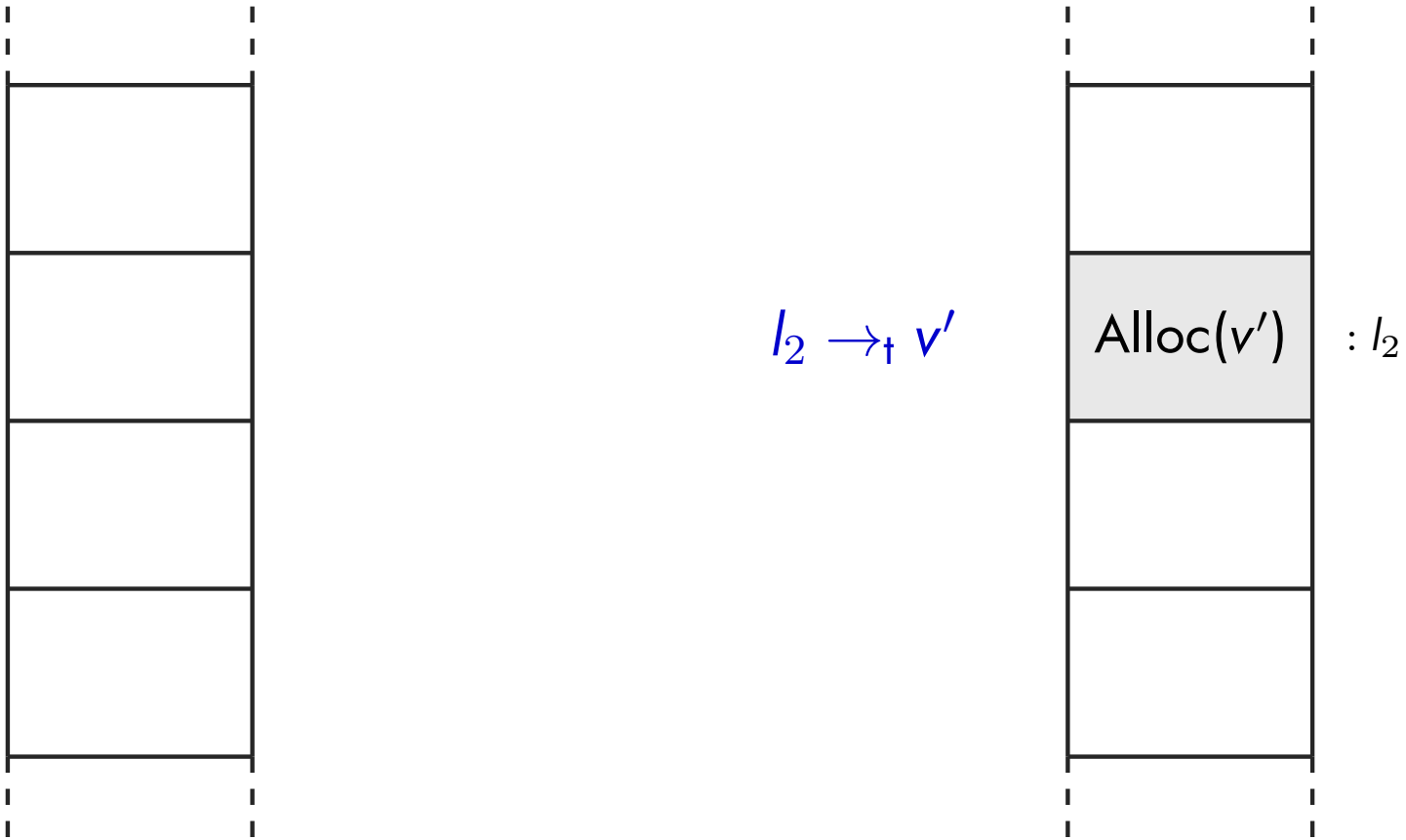
Target Heap



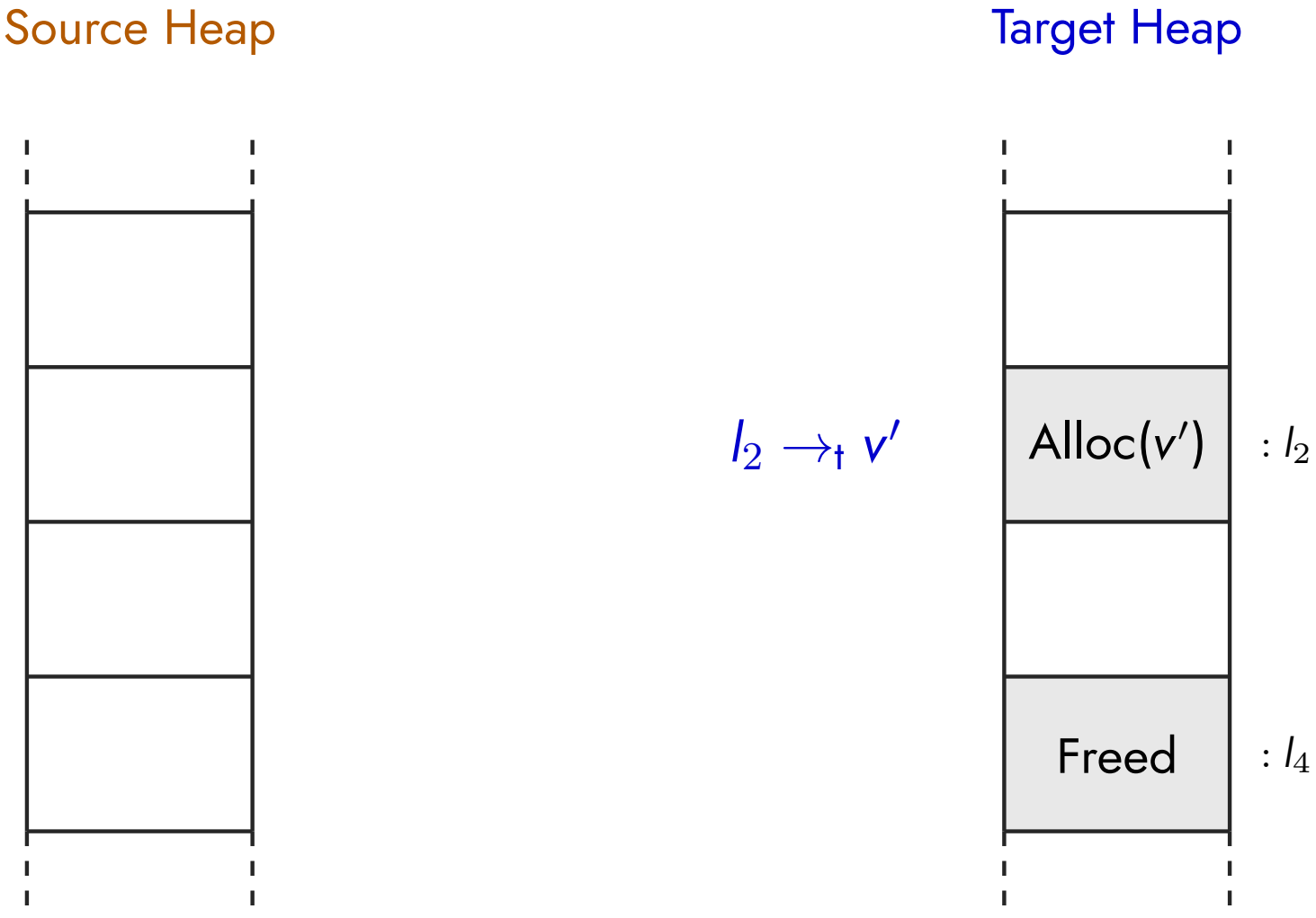
Relational Separation Logic

Source Heap

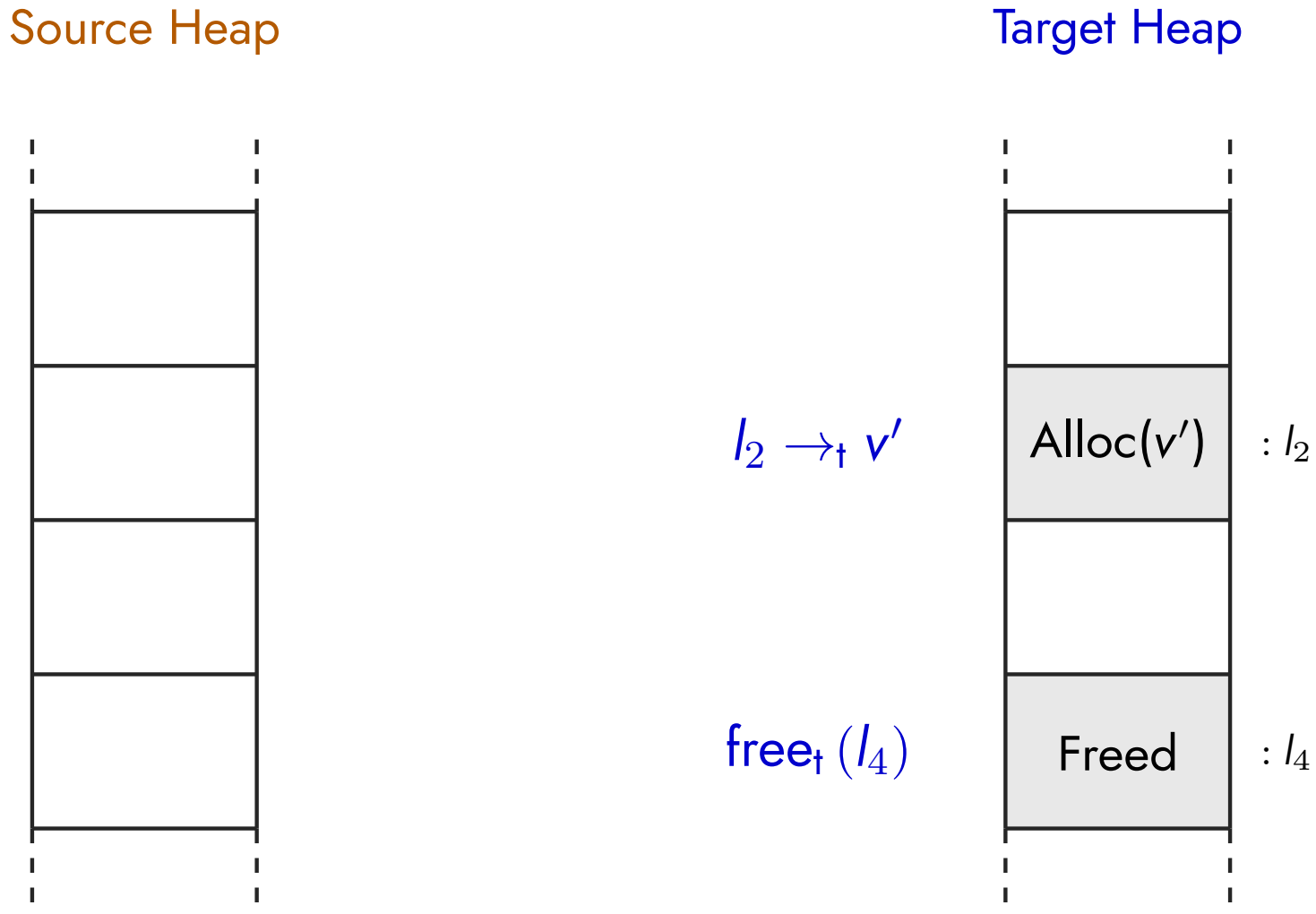
Target Heap



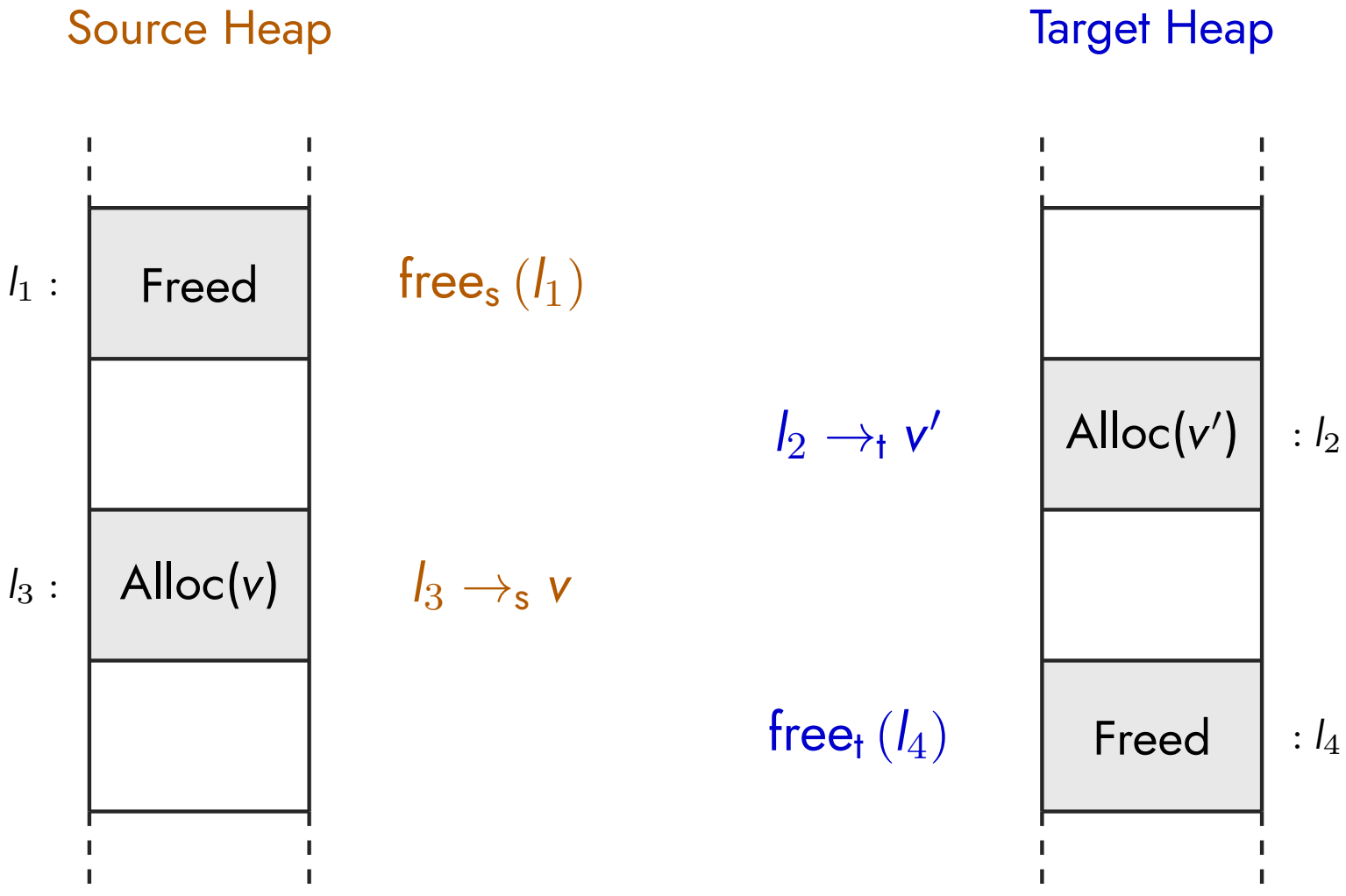
Relational Separation Logic



Relational Separation Logic

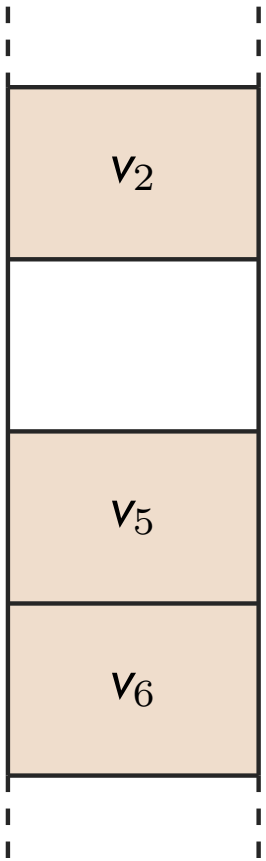


Relational Separation Logic

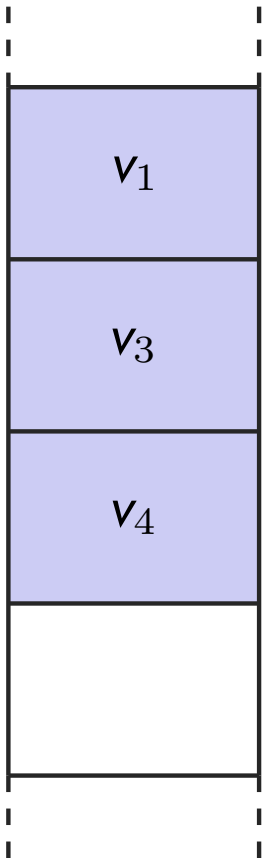


Value Equivalence

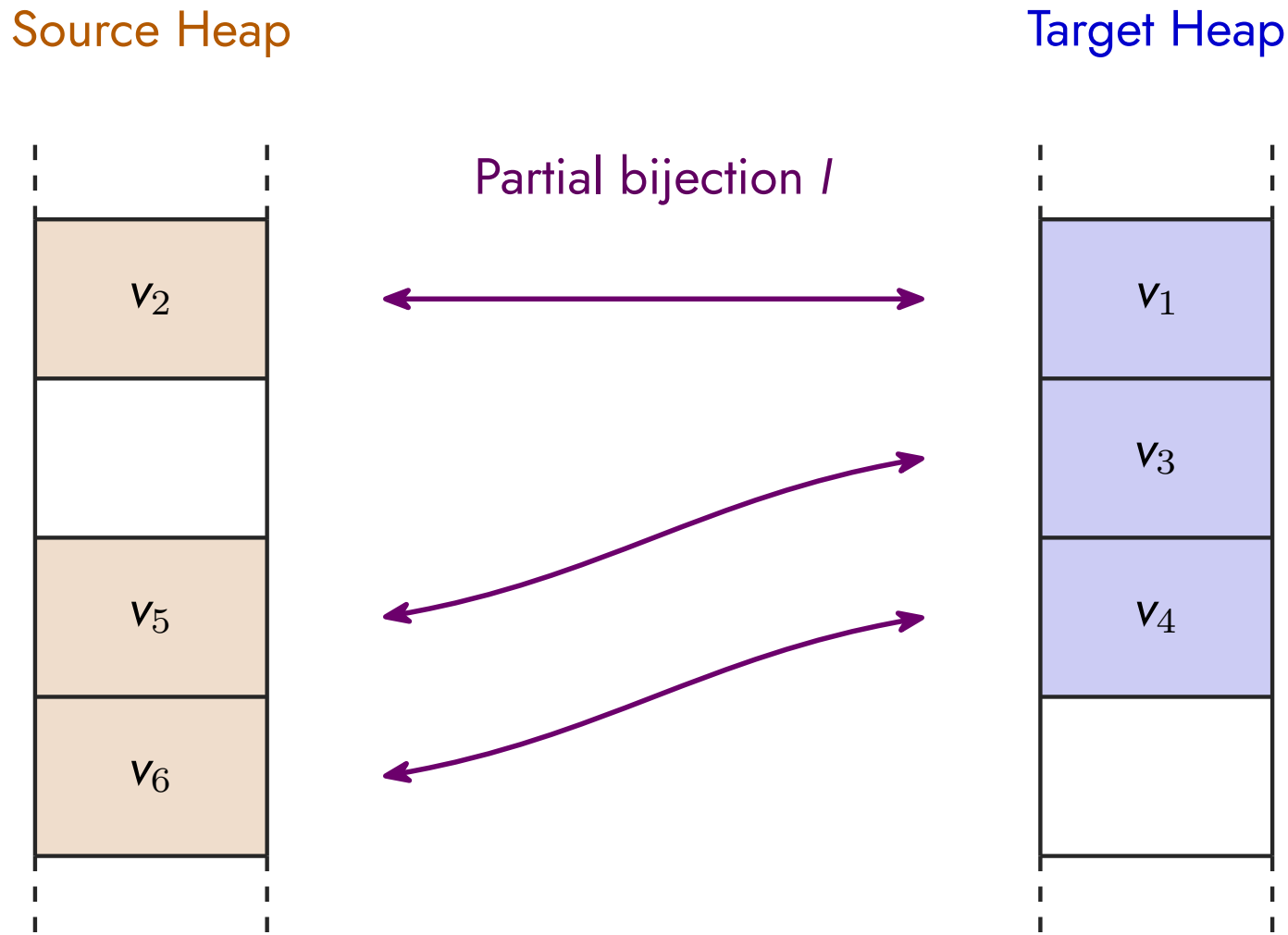
Source Heap



Target Heap



Value Equivalence



Value Equivalence

$$V_t \bowtie_I V_s$$

Value Equivalence

$$V_t \bowtie_I V_s$$

$$i_t = i_s \quad \Longrightarrow \quad \mathsf{VInt}\,(i_t) \bowtie_I \mathsf{VInt}\,(i_s)$$

Value Equivalence

$$v_t \bowtie_l v_s$$

$$i_t = i_s \implies \mathsf{VInt}\,(i_t) \bowtie_l \mathsf{VInt}\,(i_s)$$

$$b_t = b_s \implies \mathsf{VBool}\,(b_t) \bowtie_l \mathsf{VBool}\,(b_s)$$

Value Equivalence

$$v_t \bowtie_I v_s$$

$$i_t = i_s \implies \mathsf{VInt}\,(i_t) \bowtie_I \mathsf{VInt}\,(i_s)$$

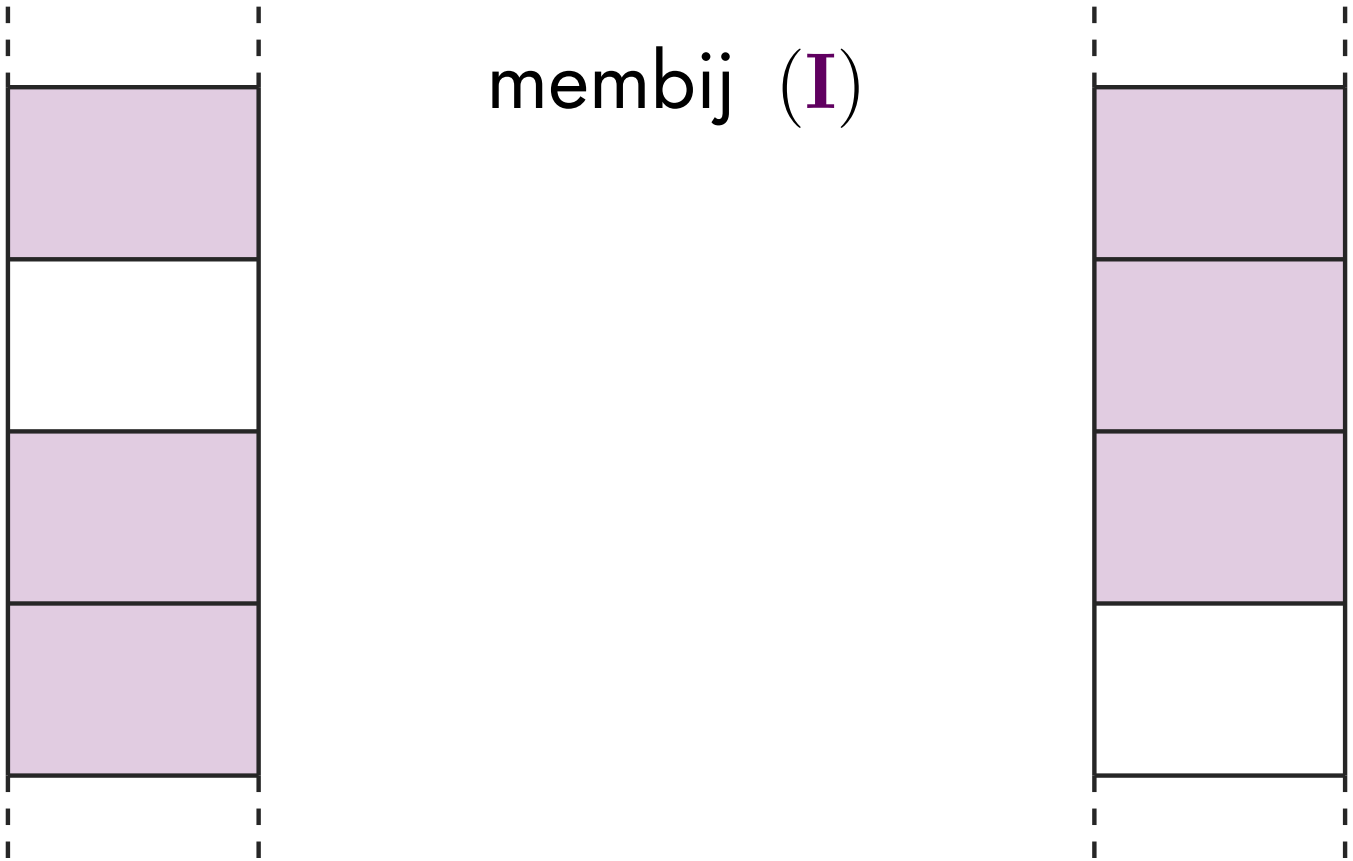
$$b_t = b_s \implies \mathsf{VBool}\,(b_t) \bowtie_I \mathsf{VBool}\,(b_s)$$

$$(l_t, l_s) \in I \implies \mathsf{VPtr}\,(l_t) \bowtie_I \mathsf{VPtr}\,(l_s)$$

Memory Bijection

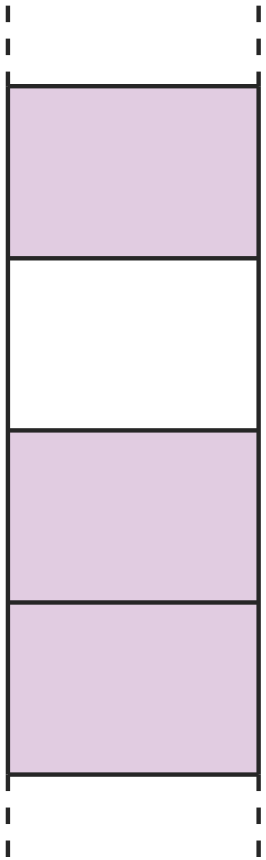
Source Heap

Target Heap



Memory Bijection

Source Heap



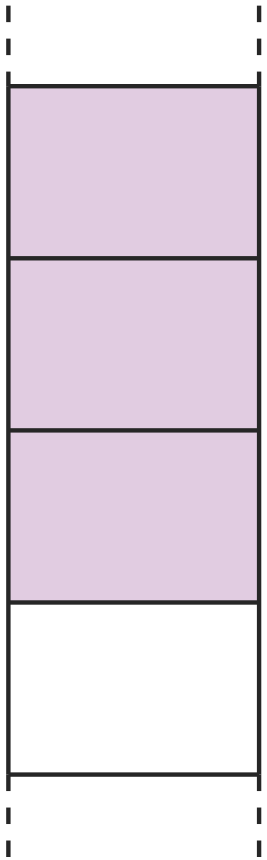
membij $(\mathbf{I})$

$(l_t, l_s) \in I$

$\Downarrow$

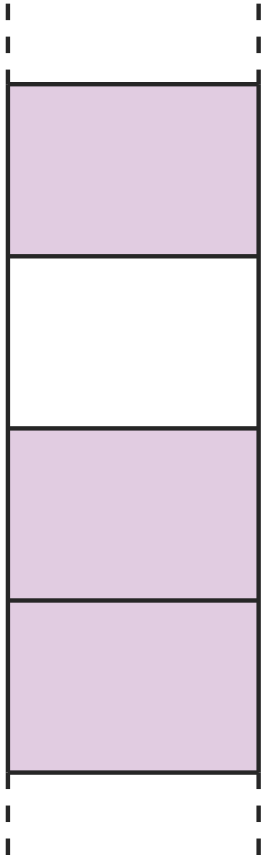
both alive with $\bowtie_I$

Target Heap

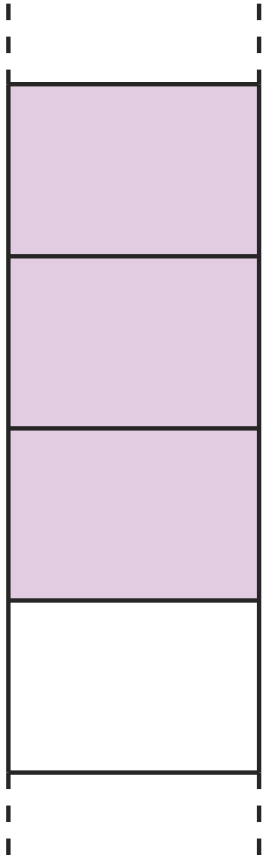


Memory Bijection

Source Heap



Target Heap



membij $(\mathbf{I})$

$(l_t, l_s) \in I$

$\Downarrow$

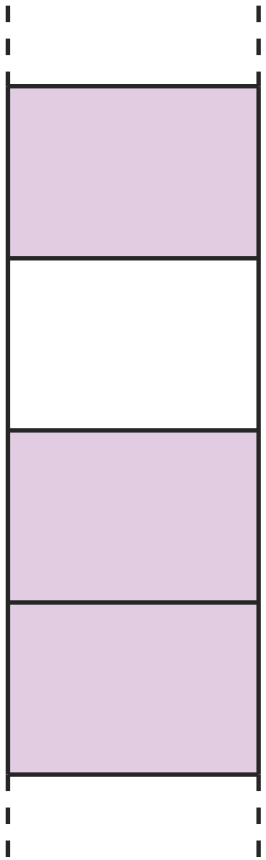
both alive with $\bowtie_I$

or

both freed

Memory Bijection

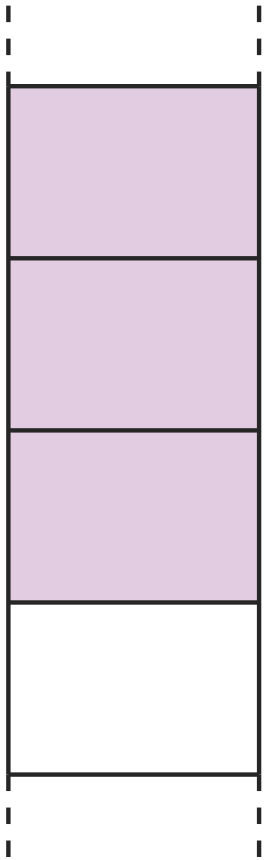
Source Heap



$\text{membij}_{\mathbf{E}}(\mathbf{I})$

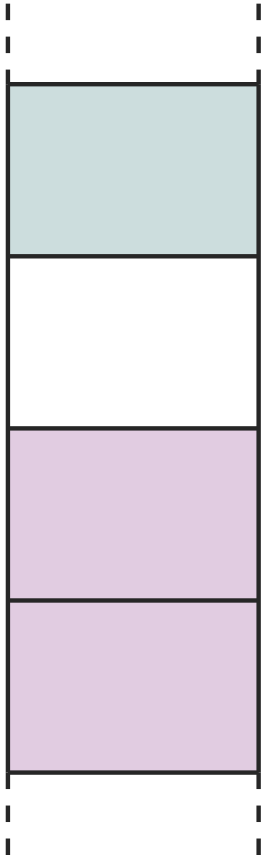
Exceptions

Target Heap



Memory Bijection

Source Heap

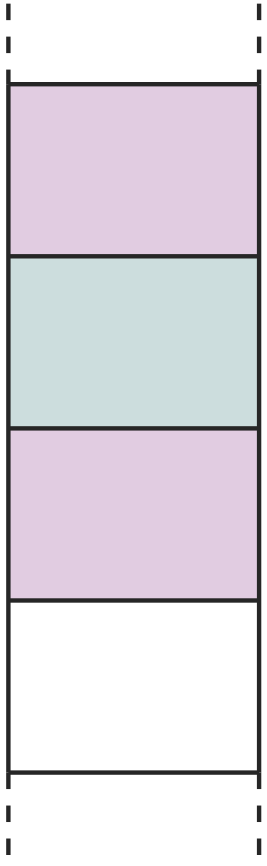


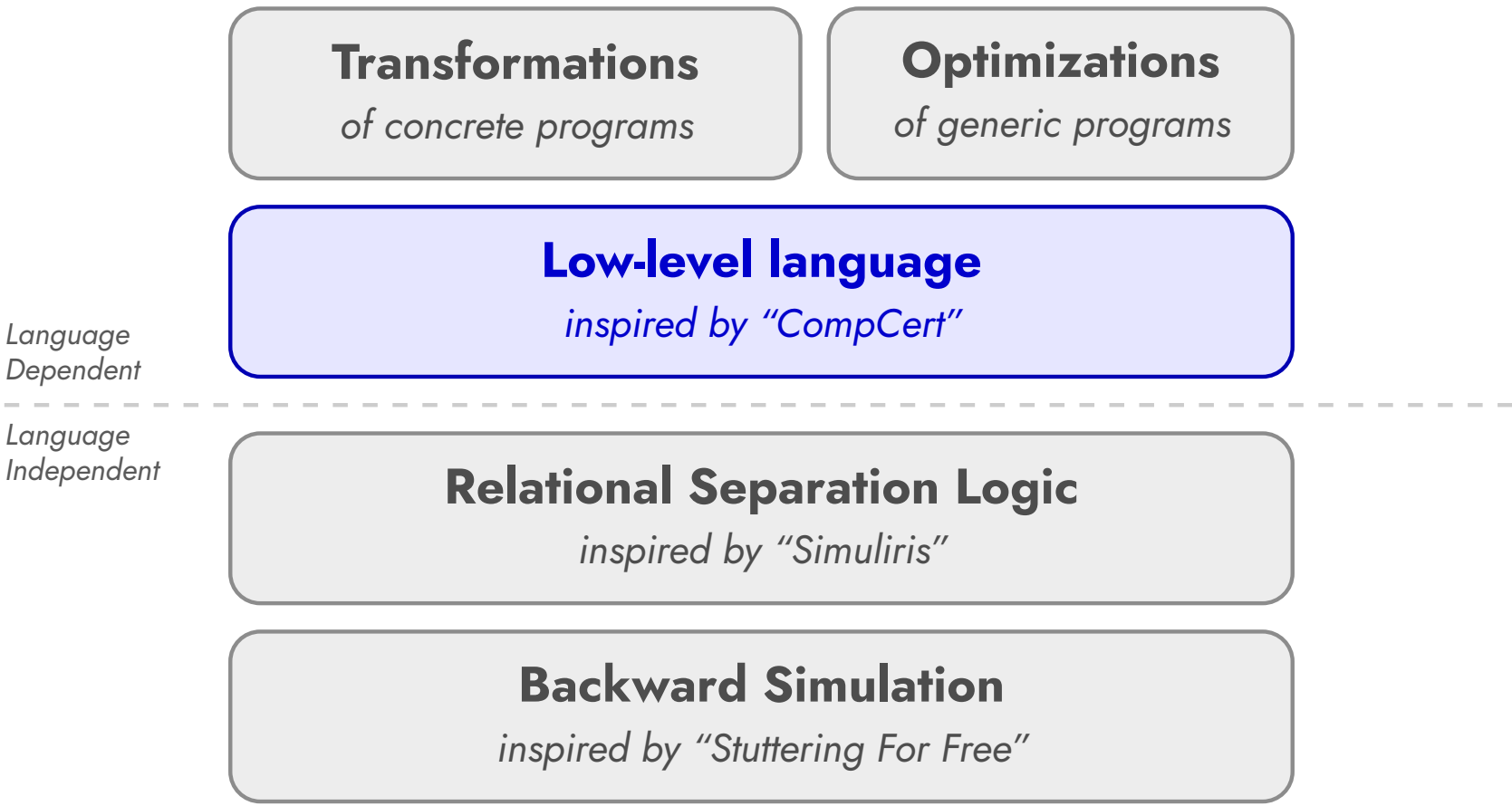
$\mathrm{membij}_{\mathbf{E}}(\mathbf{I})$

Exceptions

$E \subseteq I$

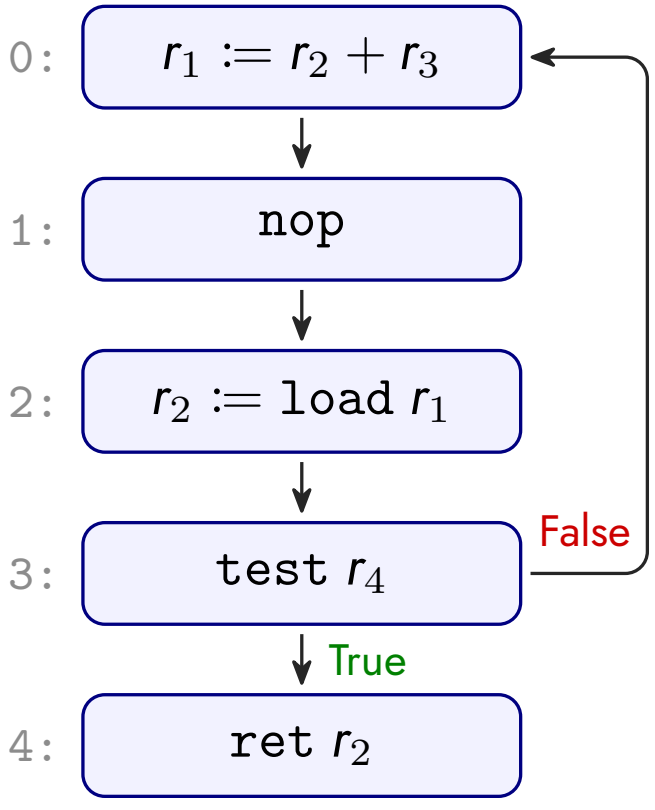
Target Heap




Transformations
of concrete programs
Optimizations
of generic programs
Low-level language
inspired by "CompCert"
Language Dependent
Language Independent
Relational Separation Logic
inspired by "Simuliris"
Backward Simulation
inspired by "Stuttering For Free"

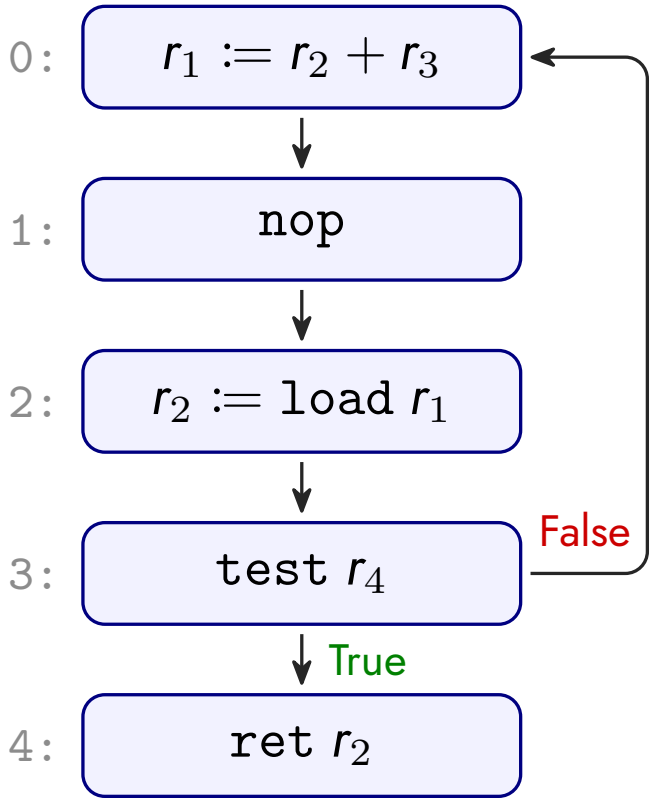
μRTL

μRTL



Function body

μRTL



Function body

Local Registers

r_1 r_2

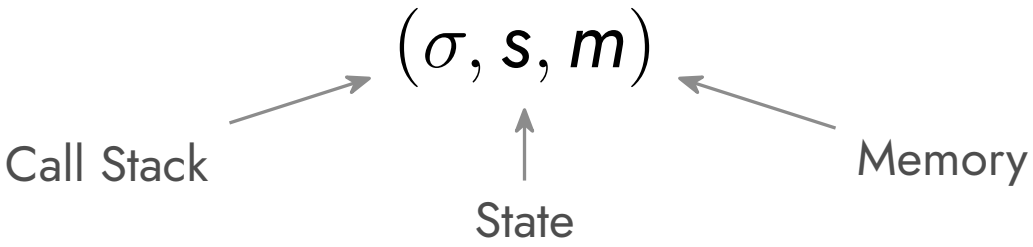
r_3 r_4

Operational Semantics

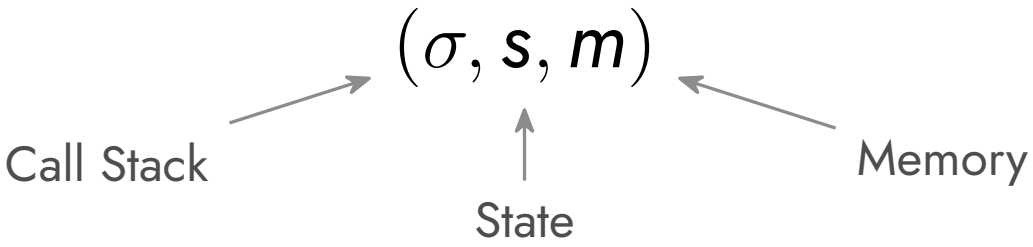
Operational Semantics

$$(\sigma, s, m)$$

Operational Semantics

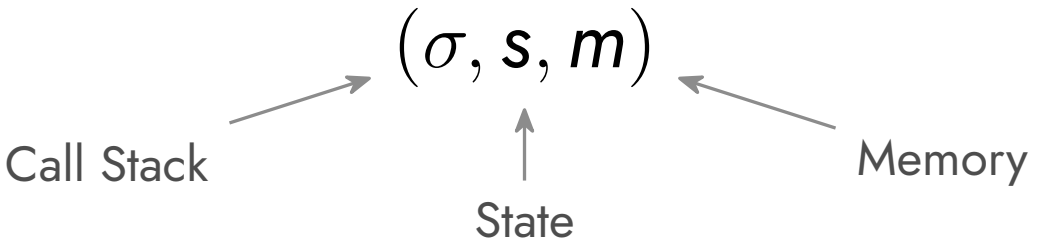


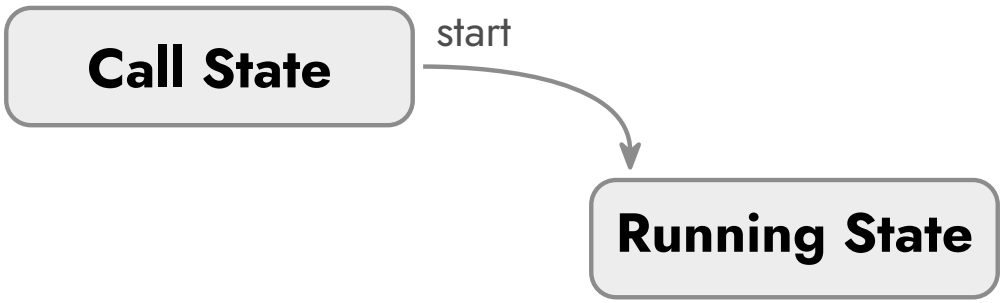
Operational Semantics



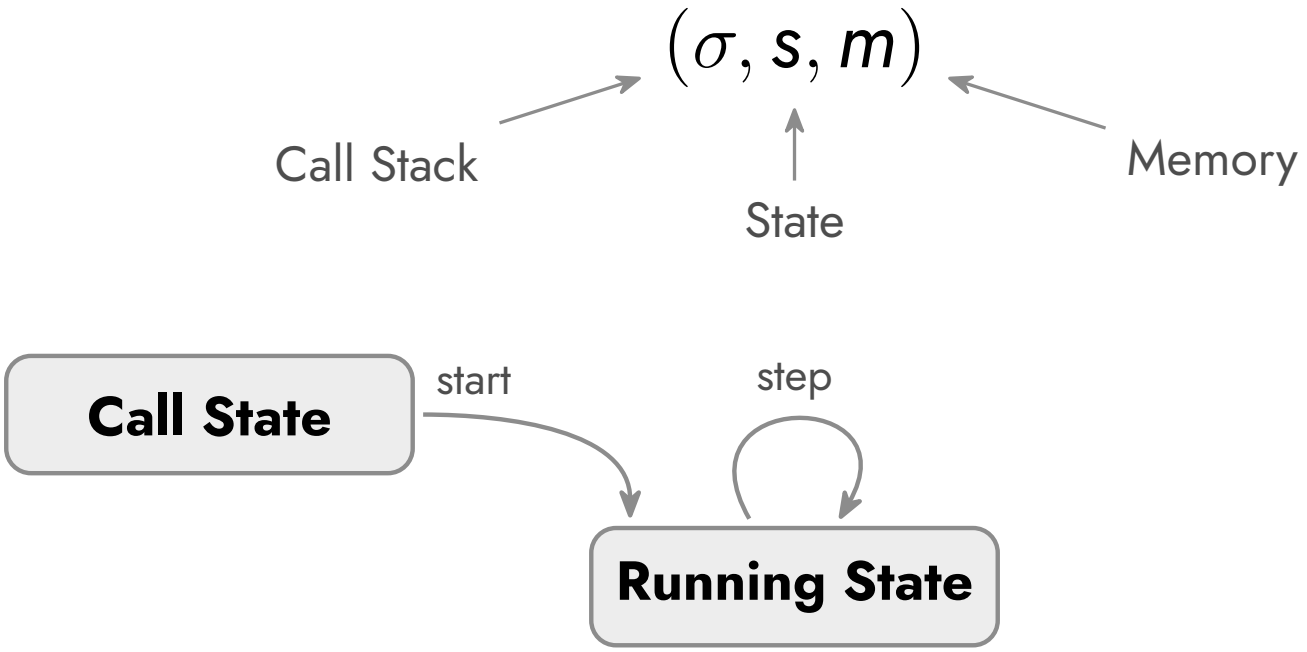
Call State

Operational Semantics

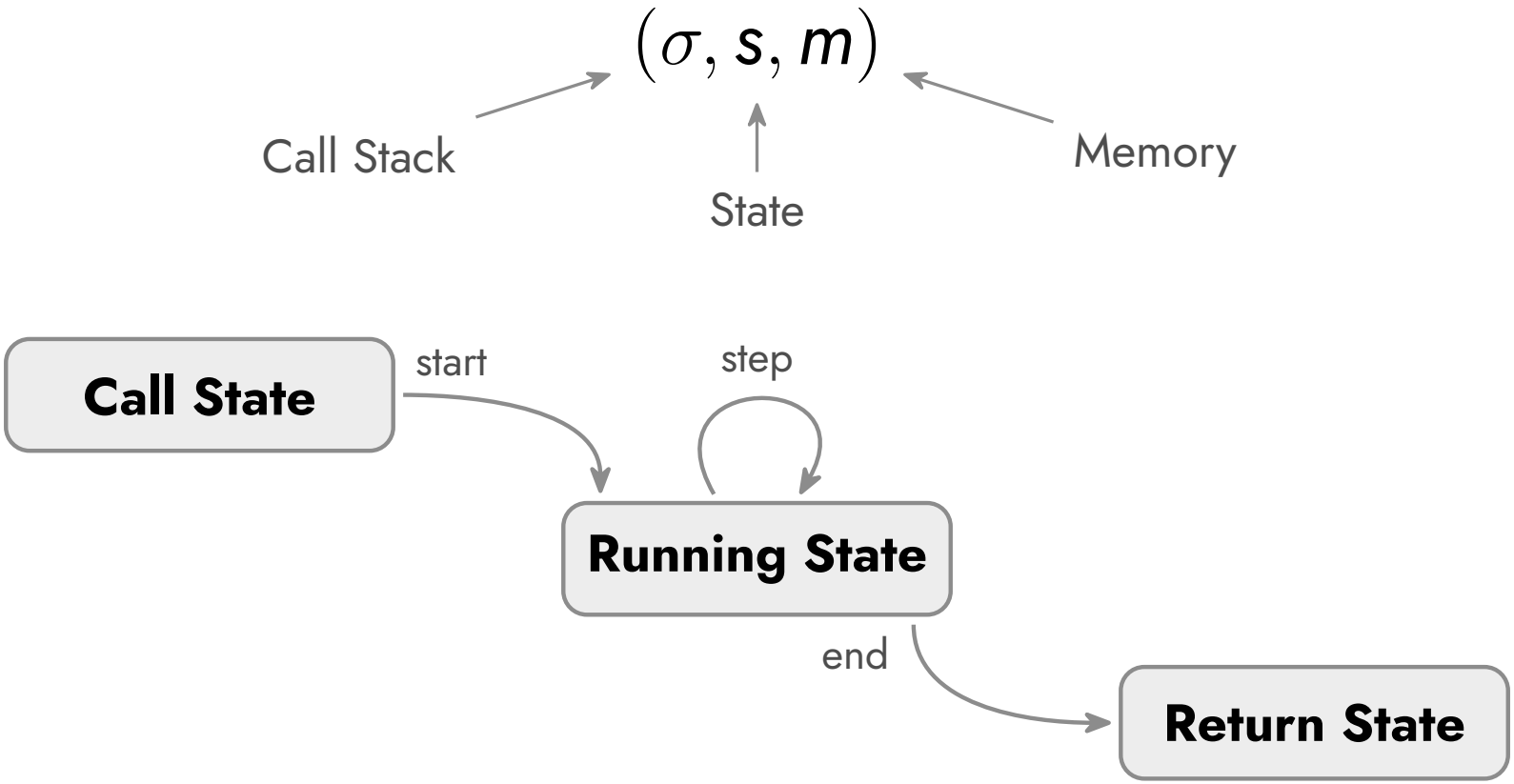




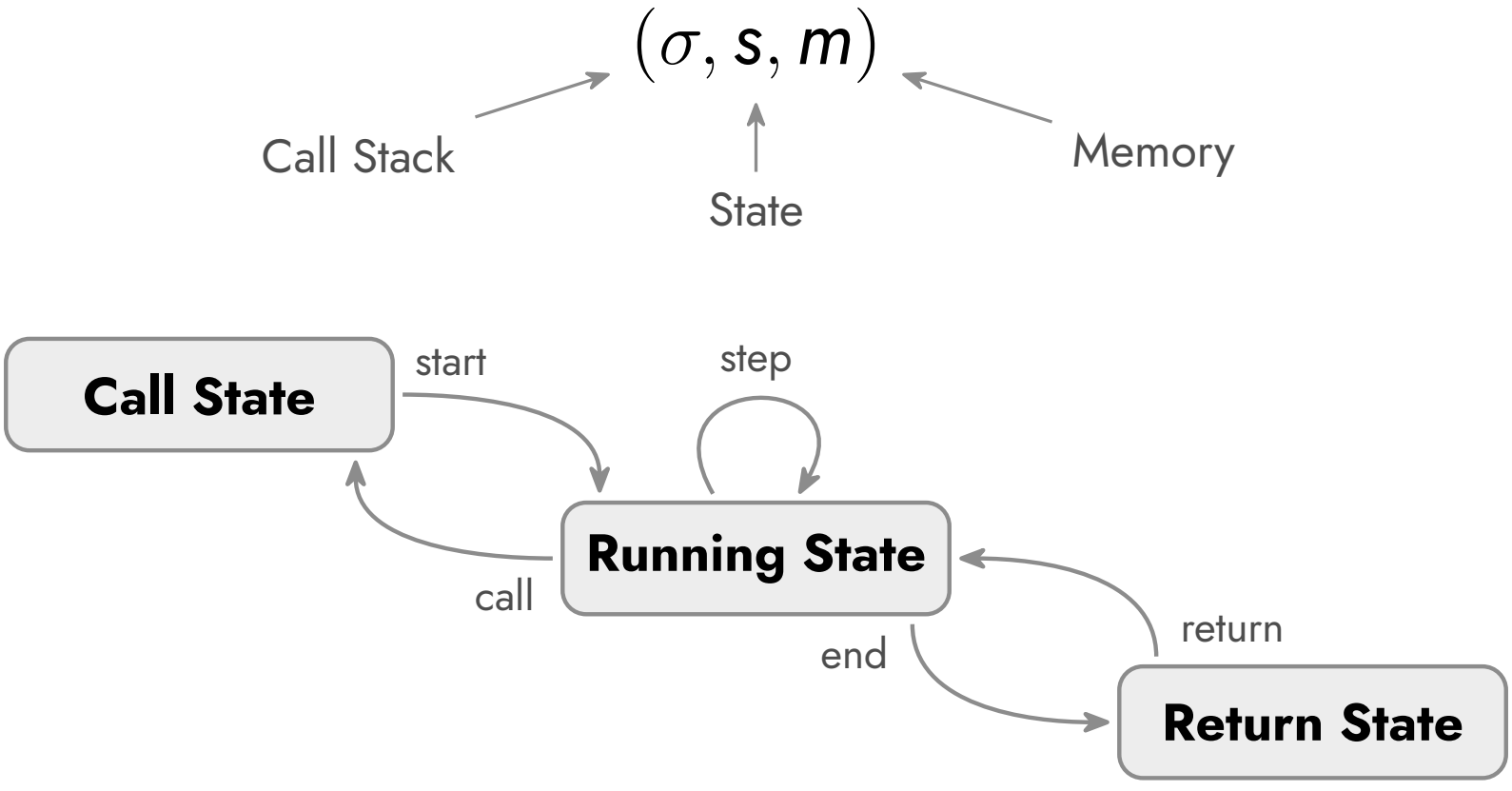
Operational Semantics



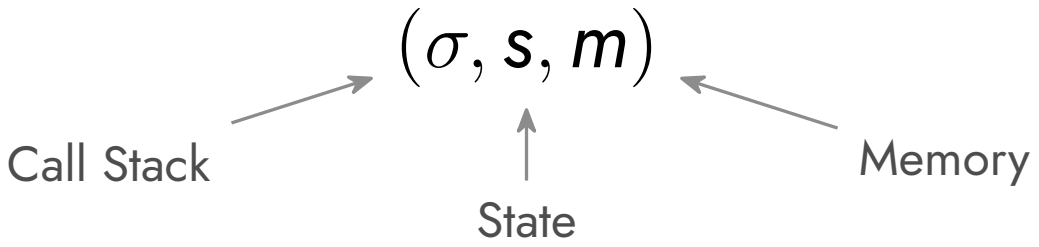
Operational Semantics

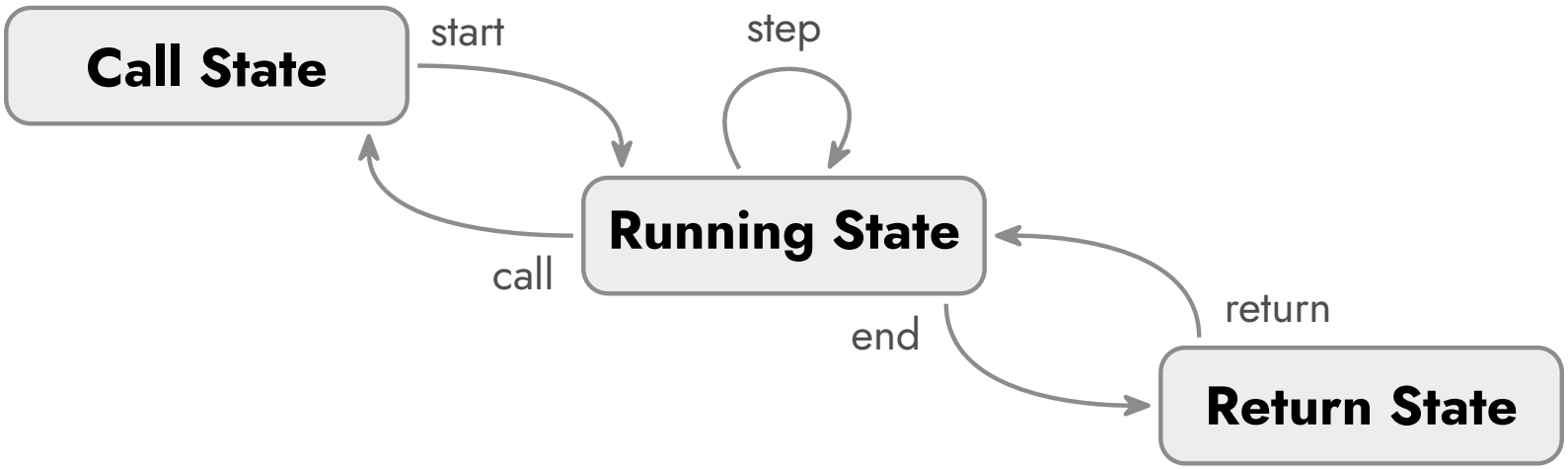


Operational Semantics



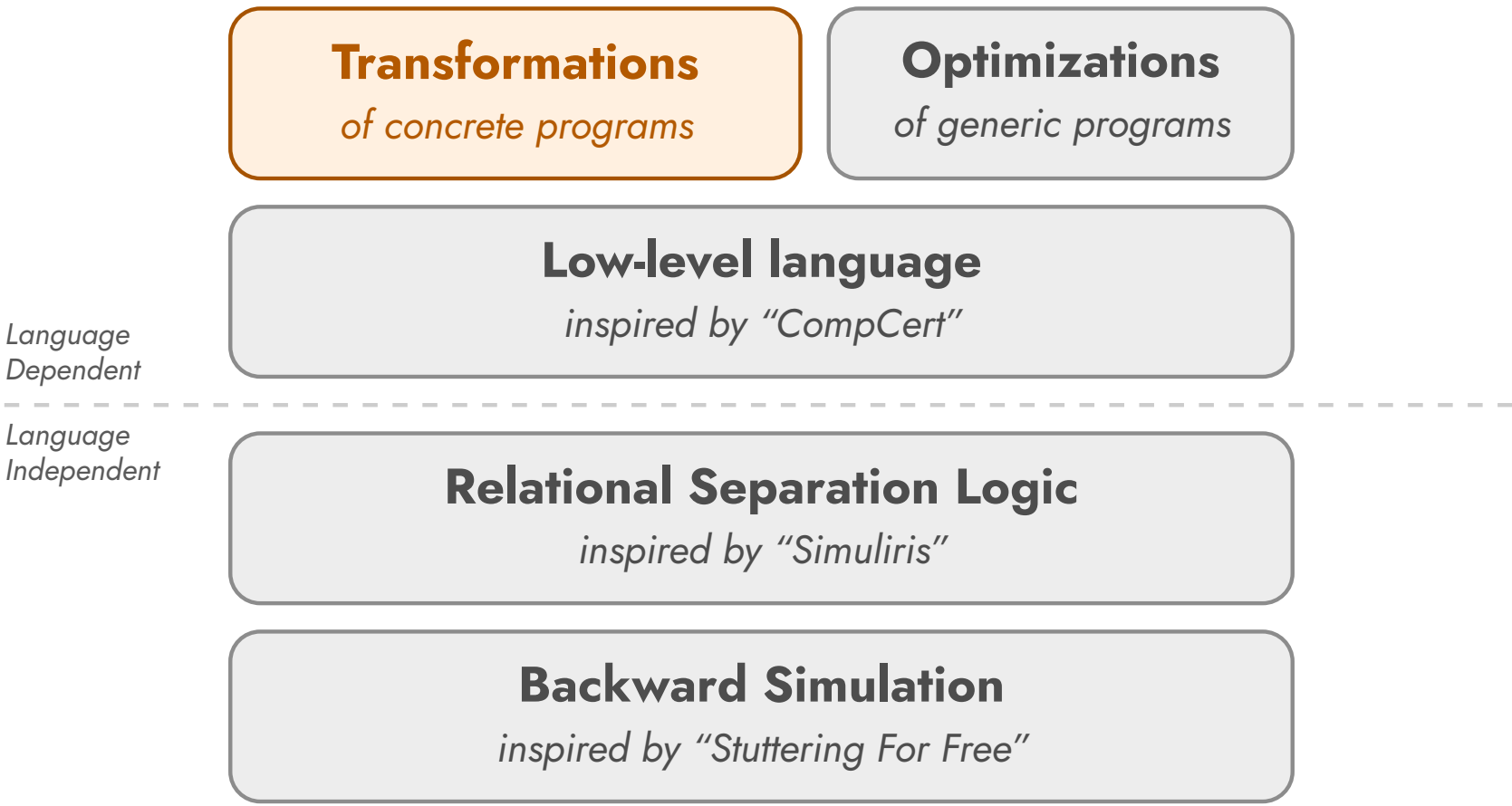
Operational Semantics





Modularity:

$$([], s, m) \twoheadrightarrow ([], s', m') \implies (\sigma, s, m) \twoheadrightarrow (\sigma, s', m')$$


Transformations
of concrete programs
Optimizations
of generic programs
Low-level language
inspired by "CompCert"
Language Dependent
Language Independent
Relational Separation Logic
inspired by "Simuliris"
Backward Simulation
inspired by "Stuttering For Free"

Eliminating a Redundant Zero-Check

Eliminating a Redundant Zero-Check

Source Code

$1:\quad \ldots \rightarrow 2$

$2:\quad r_1 := r_n / r_d \rightarrow 3$

$3:\quad r_2 := \texttt{test}\ r_d \rightarrow 4$

$4:\quad \ldots$

Eliminating a Redundant Zero-Check

Source Code

1 : $\ldots \rightarrow 2$

2 : $r_1 := r_n / r_d \rightarrow 3$

3 : $r_2 := \texttt{test}\ r_d \rightarrow 4$

4 : $\ldots$

Eliminating a Redundant Zero-Check

Source Code

$1:\quad \ldots \to 2$

$2:\quad r_1 \coloneqq r_n / r_d \to 3$

$3:\quad r_2 \coloneqq \texttt{test}\ r_d \to 4 \quad \Longrightarrow \quad 3:\quad r_2 \coloneqq \texttt{false} \to 4$

$4:\quad \ldots$

Eliminating a Redundant Zero-Check

	Source Code		Target Code
1 :	$\ldots \to 2$		$\ldots \to 2$
2 :	$r_1 := r_n / r_d \to 3$		$r_1 := r_n / r_d \to 3$
3 :	$r_2 := \texttt{test}\ r_d \to 4$	$\Longrightarrow$	$r_2 := \texttt{false} \to 4$
4 :	$\ldots$		$\ldots$

Eliminating a Redundant Zero-Check

Source Code	Target Code
$1:\ \ldots \to 2$	$1:\ \ldots \to 2$
$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I)\}$	
$2:\ r_1 := r_n / r_d \to 3$	$2:\ r_1 := r_n / r_d \to 3$
$3:\ r_2 := \mathtt{test}\ r_d \to 4$	$3:\ r_2 := \mathtt{false} \to 4$
$4:\ \ldots$	$4:\ \ldots$

Eliminating a Redundant Zero-Check

Source Code	Target Code
$1:\ \ldots \to 2$	$1:\ \ldots \to 2$
$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I)\}$	
$2:\ r_1 := r_n / r_d \to 3$	$2:\ r_1 := r_n / r_d \to 3$
• $r_d = 0:$ $\{\top\}$	
$3:\ r_2 := \mathtt{test}\ r_d \to 4$	$3:\ r_2 := \mathtt{false} \to 4$
$4:\ \ldots$	$4:\ \ldots$

Eliminating a Redundant Zero-Check

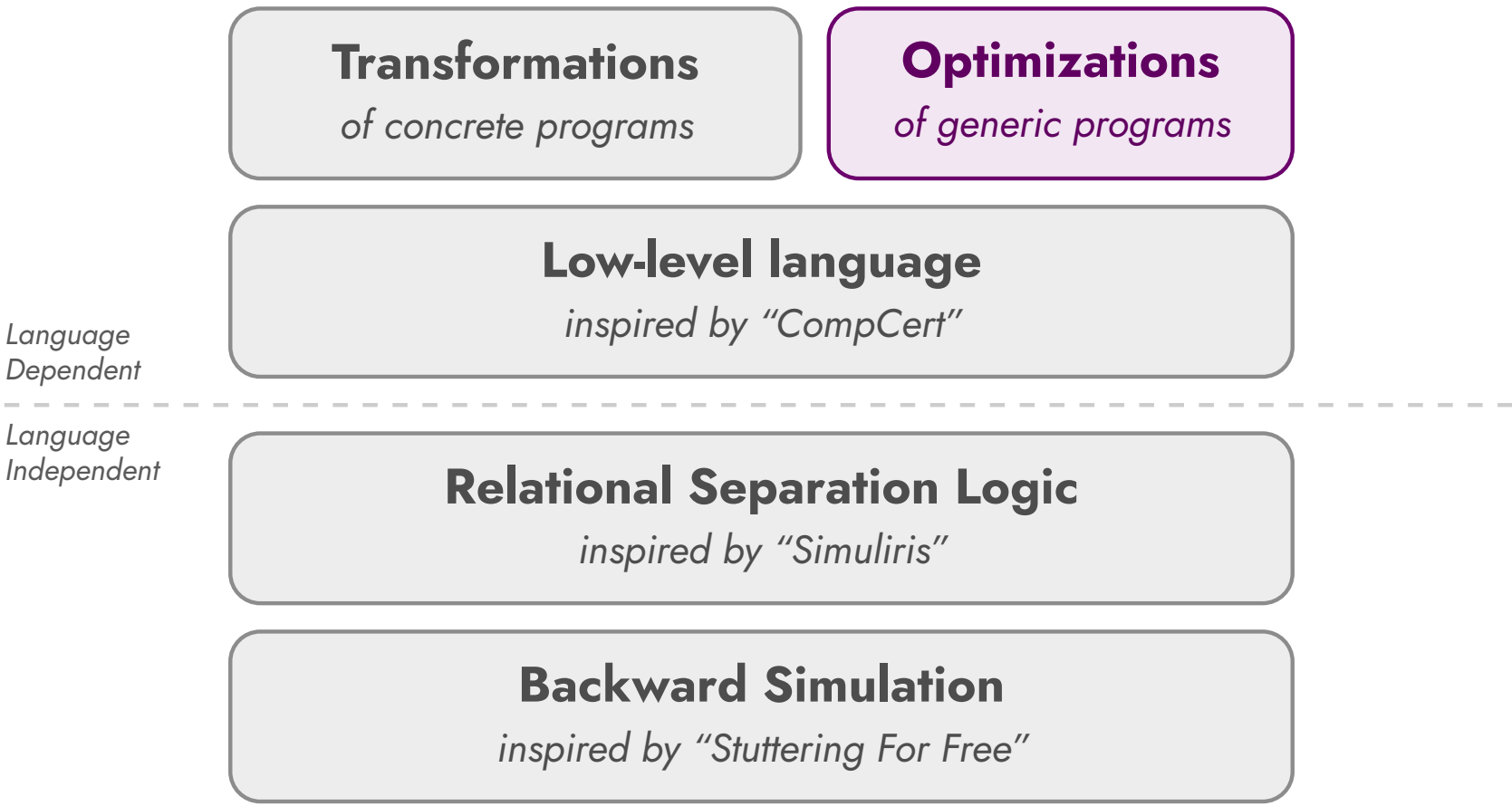
Source Code		Target Code
$1:\ \ldots \to 2$		$1:\ \ldots \to 2$
	$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I)\}$	
$2:\ r_1 := r_n / r_d \to 3$		$2:\ r_1 := r_n / r_d \to 3$
• $r_d = 0:$	$\{\top\}$	because $\lightning \in$ source
$3:\ r_2 := \texttt{test}\ r_d \to 4$		$3:\ r_2 := \texttt{false} \to 4$
$4:\ \ldots$		$4:\ \ldots$

Eliminating a Redundant Zero-Check

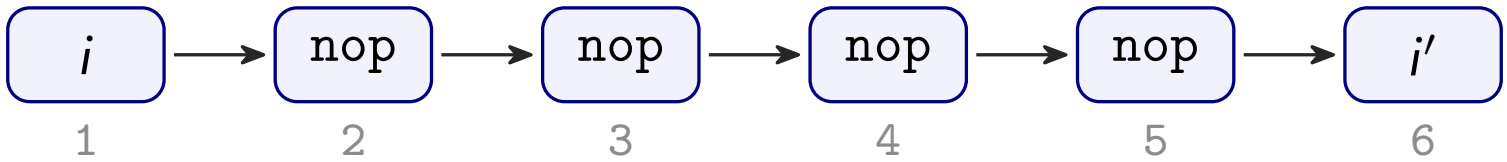
Source Code	Target Code
$1:\ \ldots \to 2$	$1:\ \ldots \to 2$
$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I)\}$	
$2:\ r_1 := r_n / r_d \to 3$	$2:\ r_1 := r_n / r_d \to 3$
• $r_d = 0$: $\{\top\}$	because $\lightning \in$ source
• $r_d \neq 0$: $\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I) * r_d \neq 0\}$	
$3:\ r_2 := \mathtt{test}\ r_d \to 4$	$3:\ r_2 := \mathtt{false} \to 4$
$4:\ \ldots$	$4:\ \ldots$

Eliminating a Redundant Zero-Check

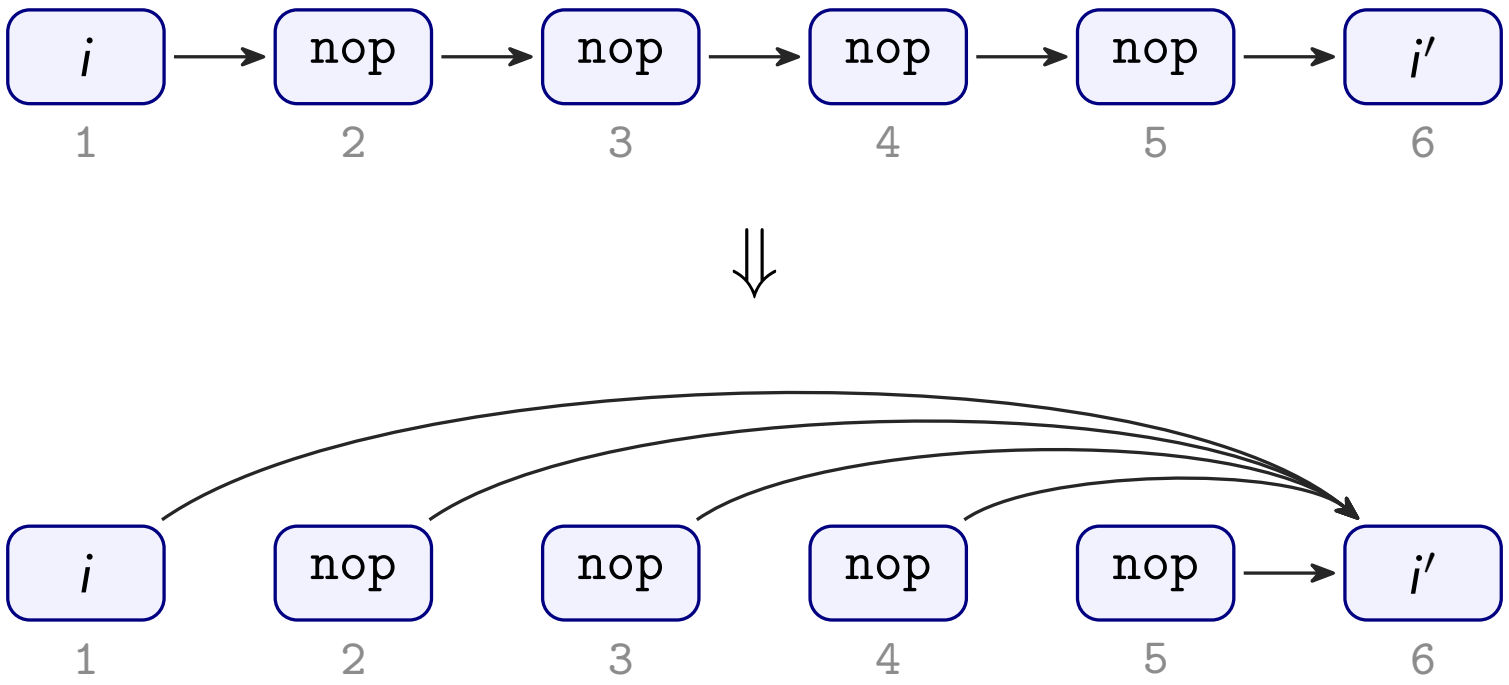
Source Code		Target Code
$1:\ \ldots \to 2$		$1:\ \ldots \to 2$
	$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I)\}$	
$2:\ r_1 := r_n / r_d \to 3$		$2:\ r_1 := r_n / r_d \to 3$
• $r_d = 0$:	$\{\top\}$	because $\lightning \in$ source
• $r_d \neq 0$:	$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I) * r_d \neq 0\}$	
$3:\ r_2 := \mathtt{test}\ r_d \to 4$		$3:\ r_2 := \mathtt{false} \to 4$
	$\{\mathsf{membij}_{\emptyset}(I) * \mathsf{same\_regs}(I)\}$	
$4:\ \ldots$		$4:\ \ldots$


Transformations
of concrete programs
Optimizations
of generic programs
Low-level language
inspired by "CompCert"
Language Dependent
Language Independent
Relational Separation Logic
inspired by "Simuliris"
Backward Simulation
inspired by "Stuttering For Free"

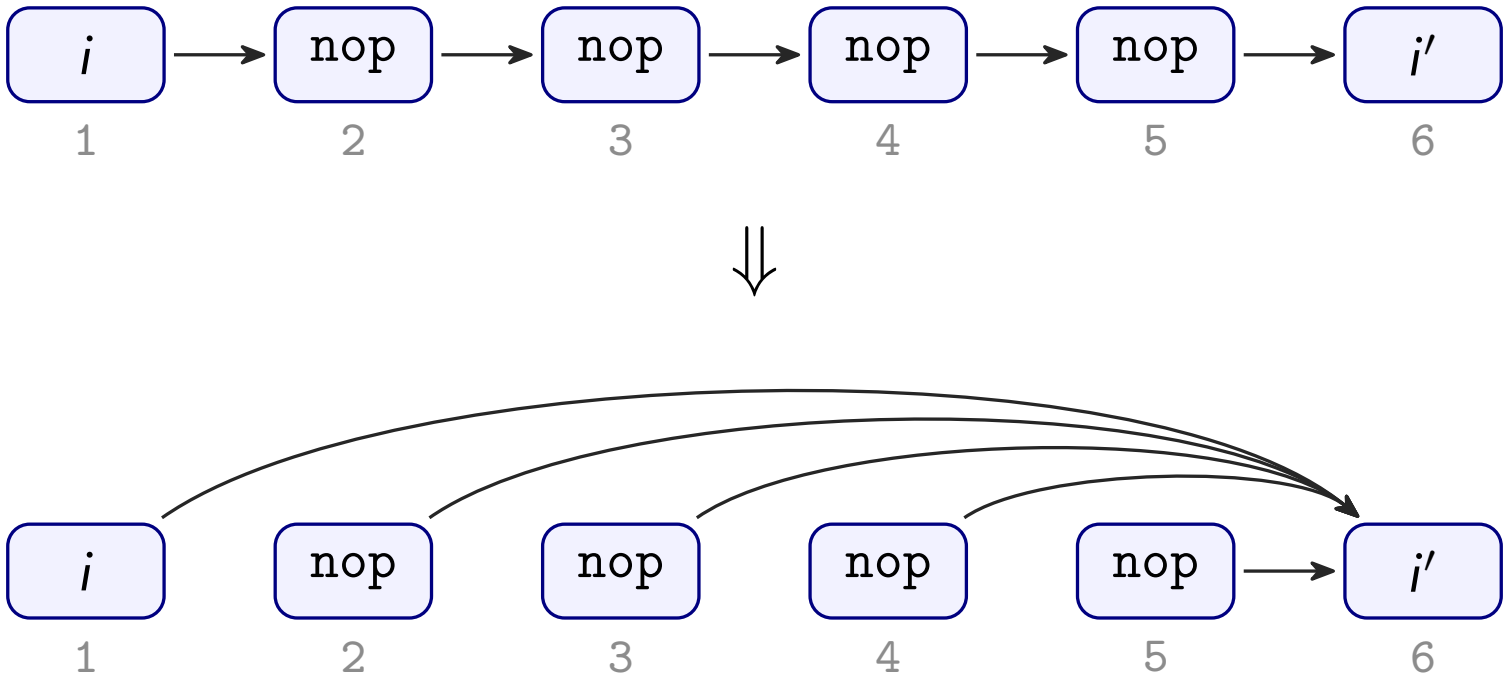
Short-Circuiting Jump Chains



Short-Circuiting Jump Chains

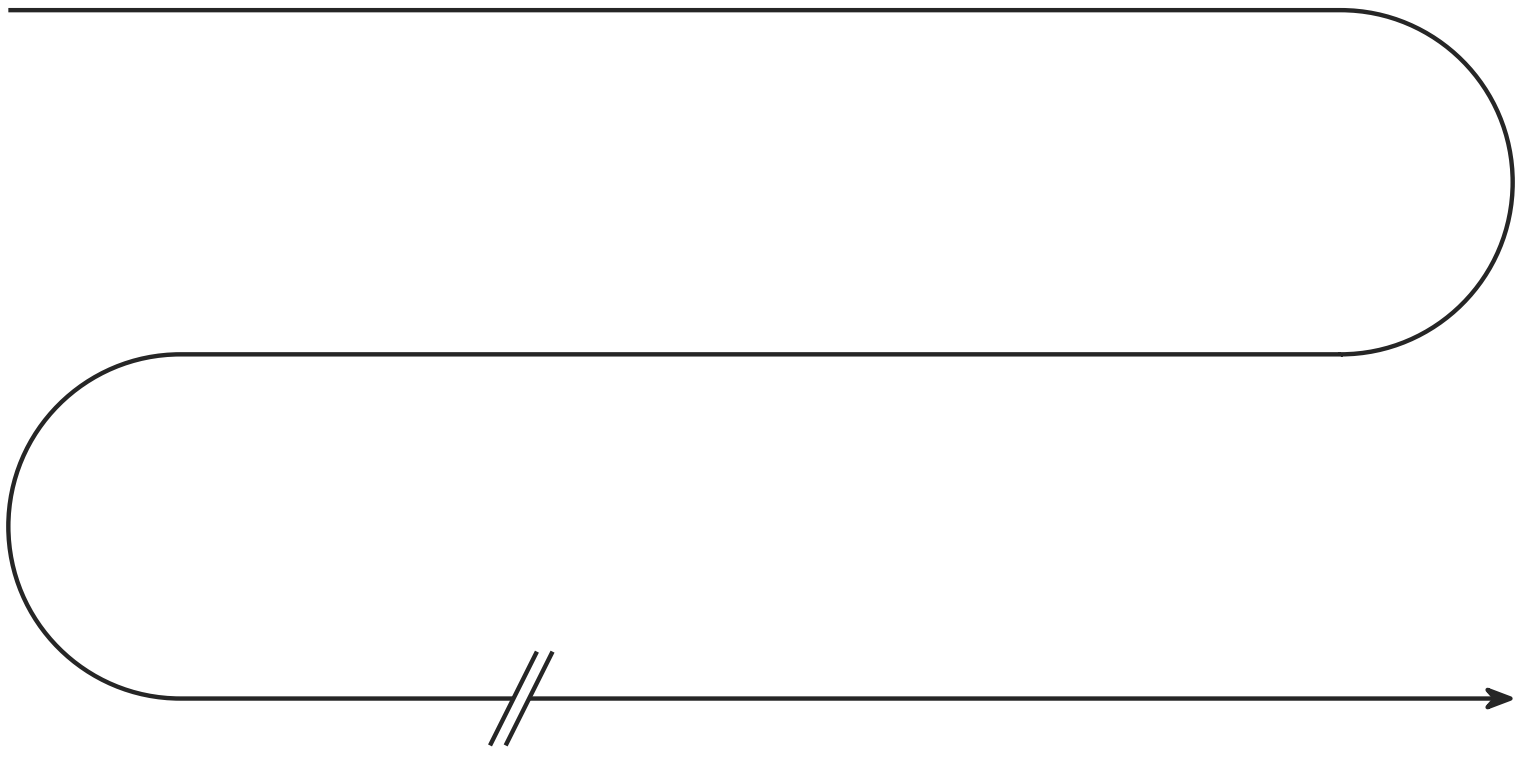


Short-Circuiting Jump Chains

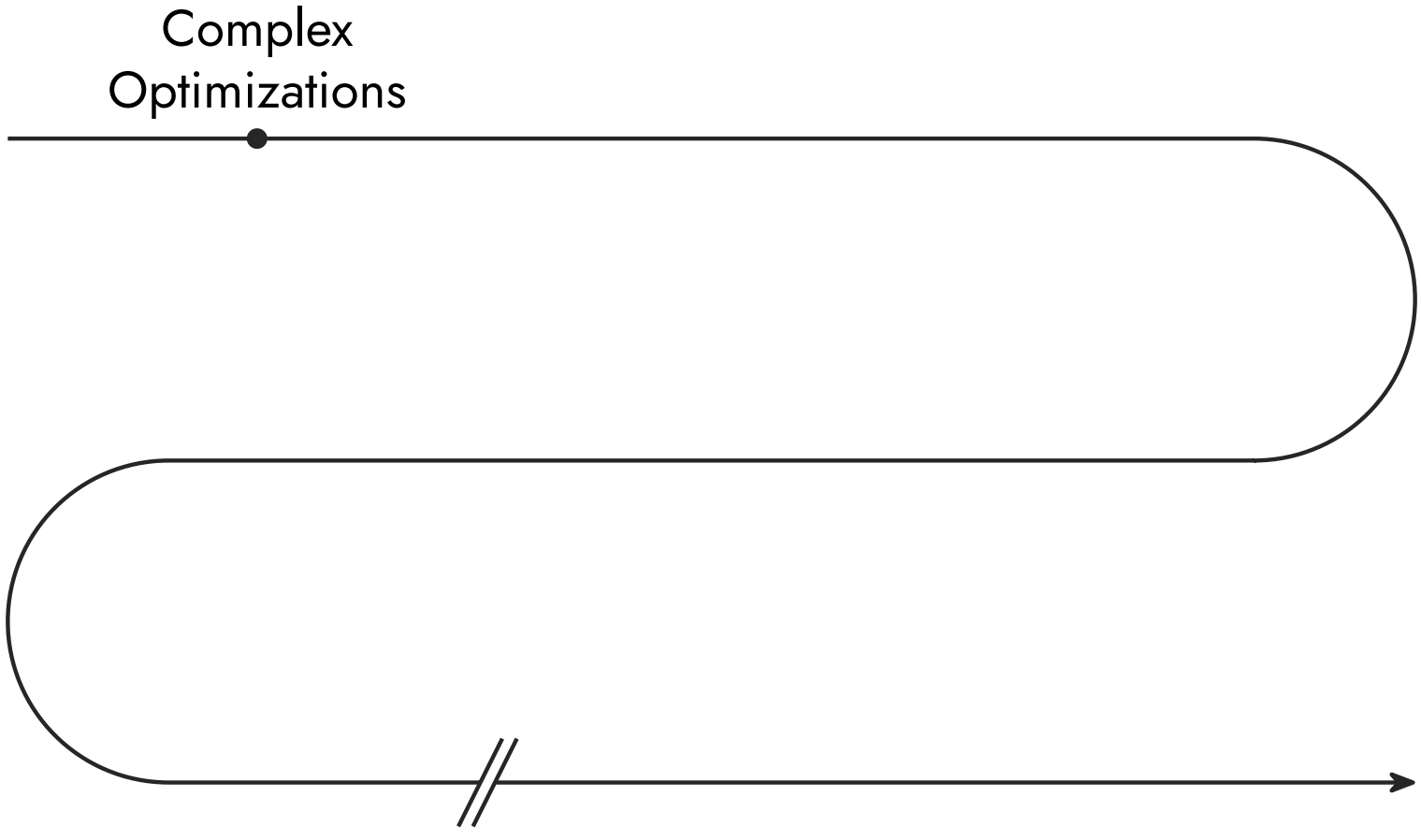


$$\{\text{same\_args}_I\}\ \text{tunnel}_d\,(f) \lesssim f\ \{\text{same\_res}_I\}$$

What Comes Next

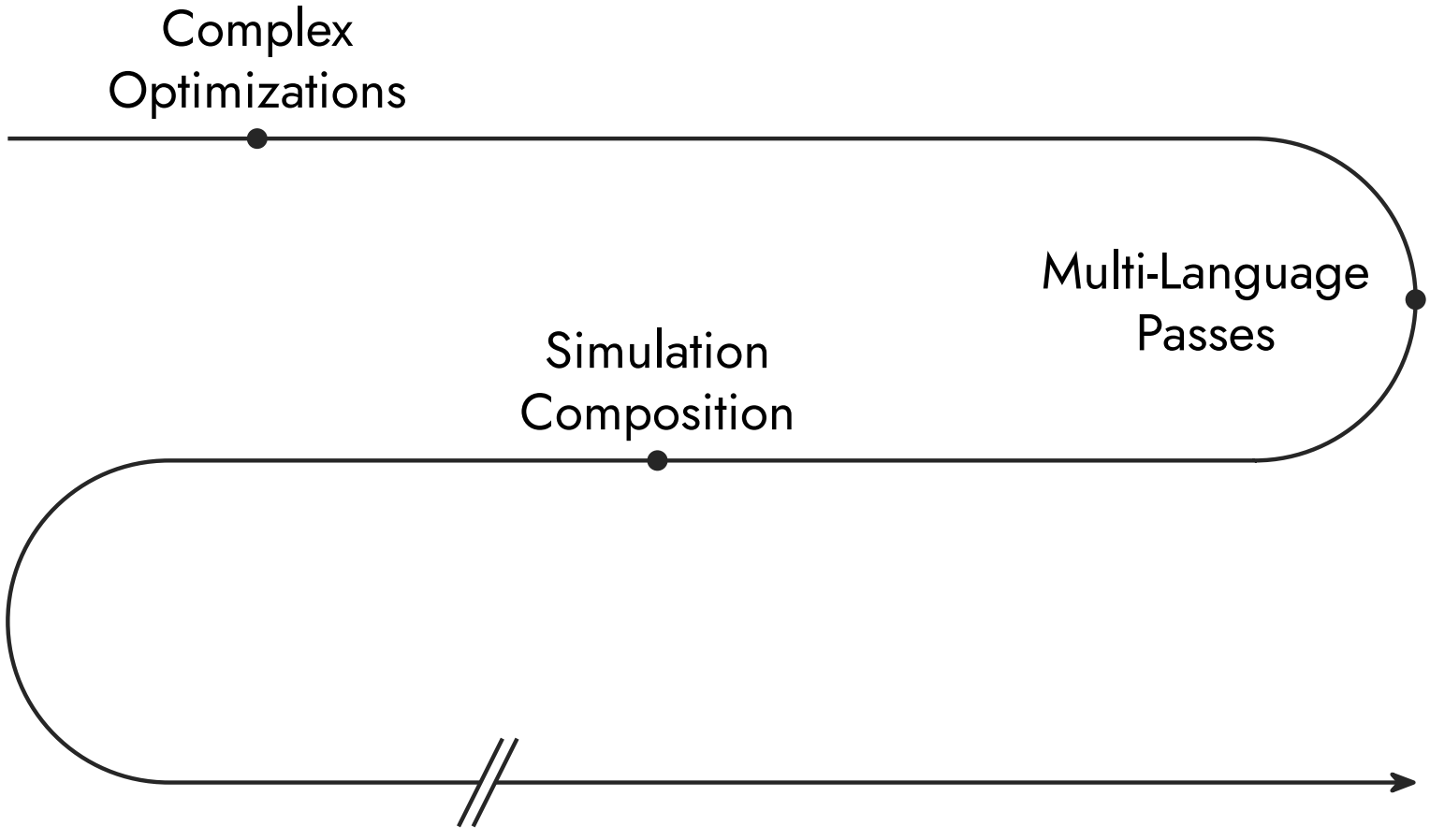


What Comes Next

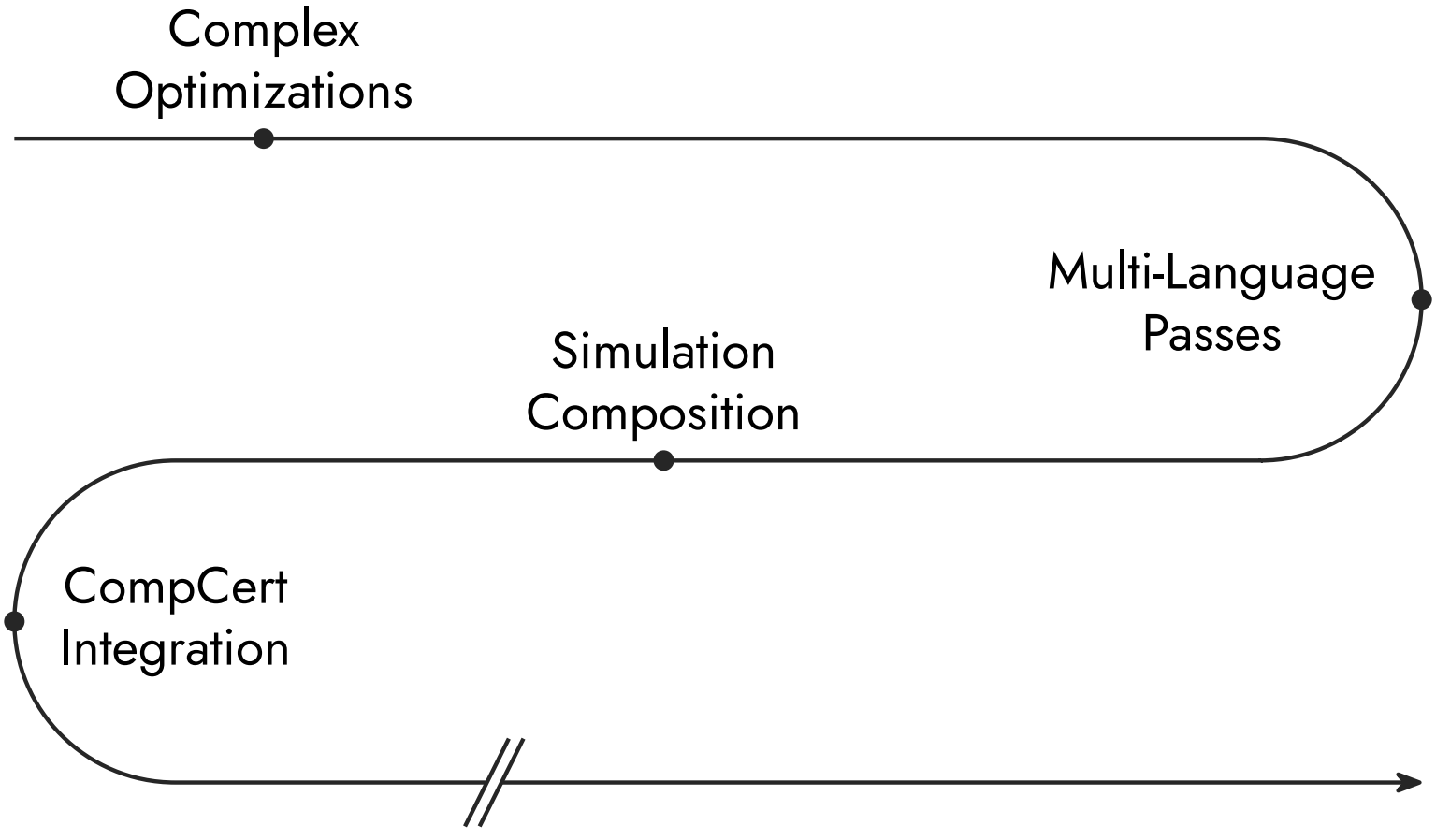


What Comes Next

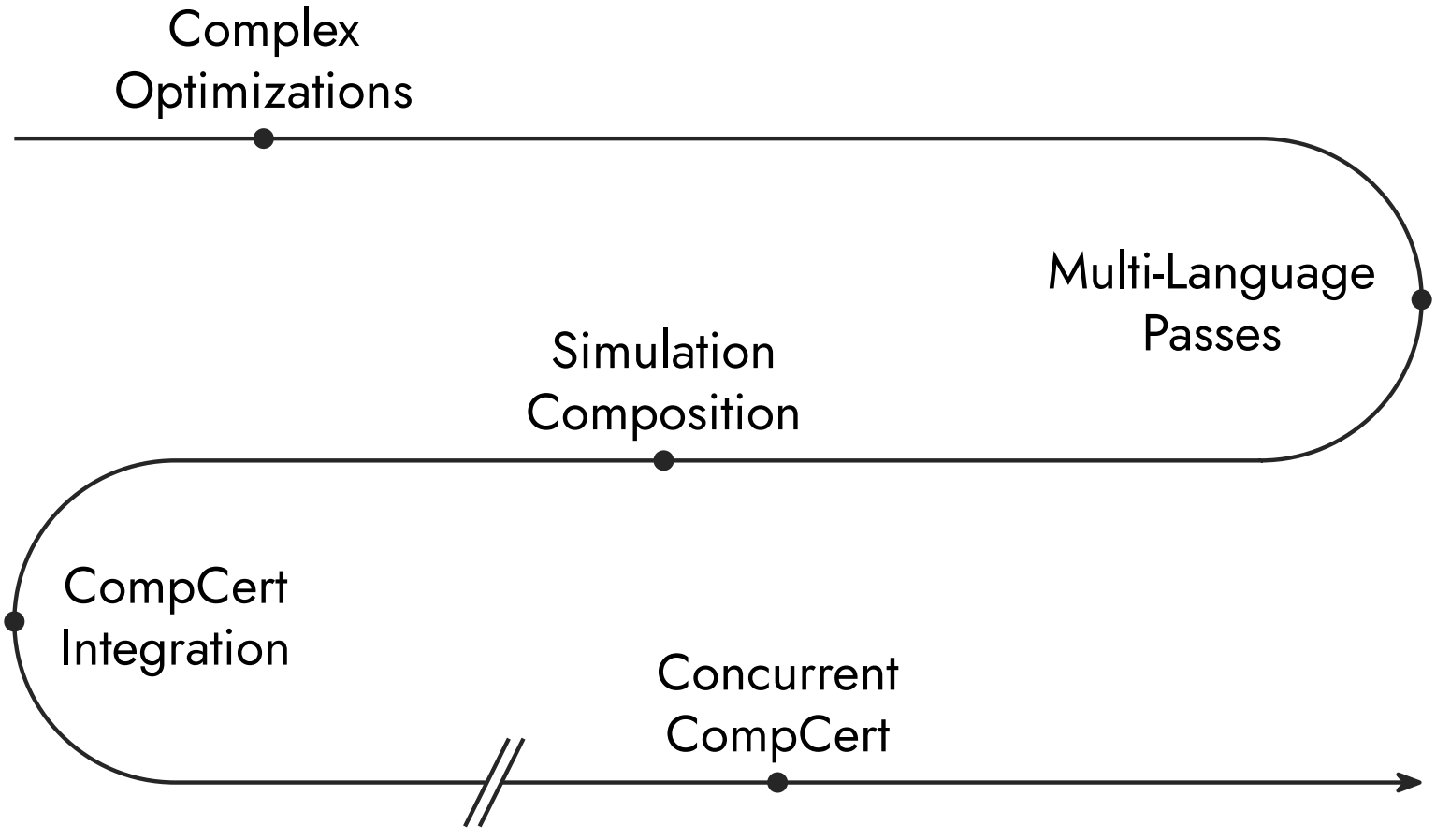
What Comes Next



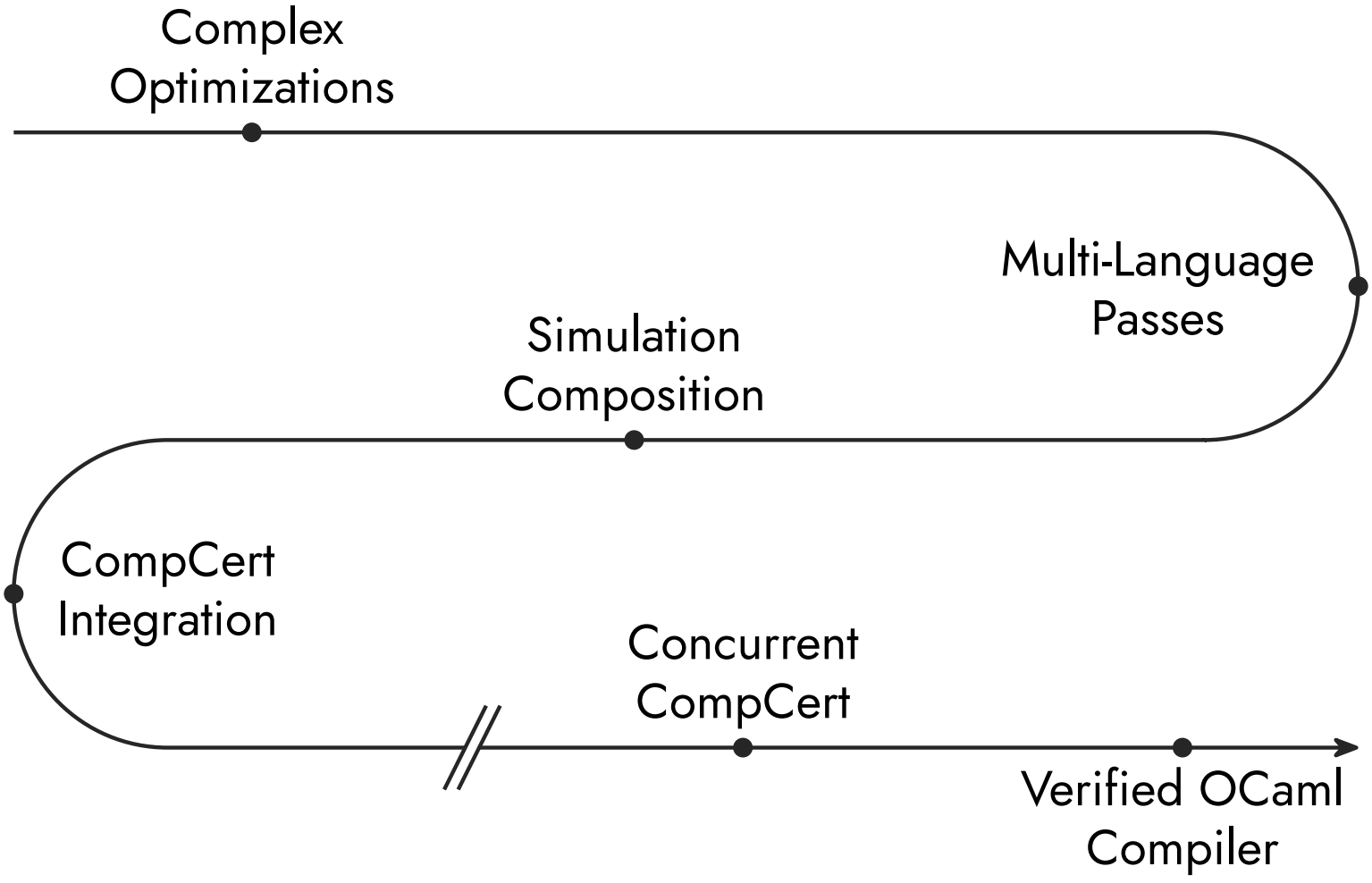
What Comes Next



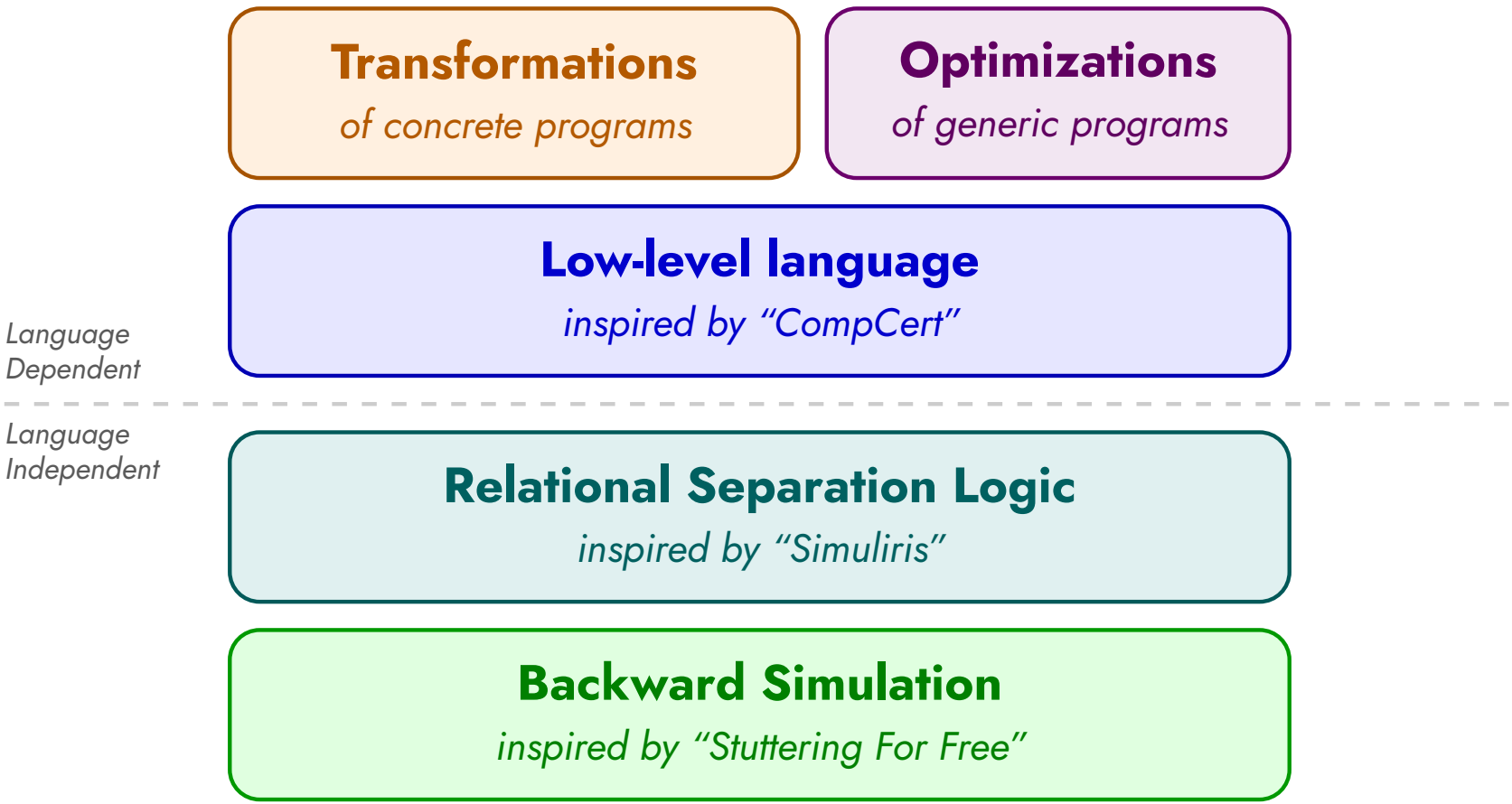
What Comes Next



What Comes Next



Thanks


Transformations
of concrete programs
Optimizations
of generic programs
Low-level language
inspired by "CompCert"
Language Dependent
Language Independent
Relational Separation Logic
inspired by "Simuliris"
Backward Simulation
inspired by "Stuttering For Free"